Innopolis University

Master's Programme in Computer Science

AI-Assisted Simulation and Optimization of Power Market Projects

Master's Thesis

Author: Dadakhon Turgunboev

Supervisor: Leonard Johard

Innopolis, 2026

Abstract

Power system simulation and electricity-market optimization underpin the planning and operation of modern grids, yet the dominant open-source framework for this domain, PyPSA (Python for Power System Analysis), faces a performance limitation: for large networks and iterative scenario studies its Python-based model-assembly layer can dominate the total runtime, no self-contained Julia reimplementation with a PyPSA-compatible interface exists, and the integration of a calibrated demand forecast into a stochastic dispatch within a single native framework remains largely unexplored. This thesis reimplements PyPSA's core computational modules in Julia and augments them with an AI-assisted forecasting layer. Six solver modules — DC power flow, AC power flow (via PowerModels.jl and Ipopt), linearized AC power flow, single- and multi-period Linear Optimal Power Flow with storage and wind, and Unit Commitment as a mixed-integer linear program — are implemented and validated against PyPSA on a 3-bus network and the IEEE 14-bus case; for the linear-programming methods the dispatch, cost, and locational marginal prices are numerically identical, and the remaining differences are explained by documented per-unit conventions. Because both implementations call the same HiGHS and Ipopt solvers, the measured speedup is isolated to the compiled model-assembly layer; benchmarks on standard networks up to 2,869 buses (IEEE and PEGASE) show the linear-programming methods running tens to a few hundred times faster than PyPSA — for example, 58$\times$ for LOPF on the 2,869-bus PEGASE grid — while AC power flow, delegated to an interior-point solver, shows no consistent advantage. An LSTM load forecaster with distribution-free conformal prediction intervals supplies demand scenarios to a sample-average-approximation stochastic dispatch that reports expected cost and conditional value-at-risk; the value of the AI layer lies in this calibrated uncertainty quantification rather than in point-forecast accuracy.

Keywords: power system optimization, Julia, PyPSA, optimal power flow, unit commitment, load forecasting, conformal prediction, stochastic optimization, locational marginal prices.

Contents

List of Figures

List of Tables

List of Abbreviations

AC	Alternating Current
ACPF	AC Power Flow
AD	Automatic Differentiation
CVaR	Conditional Value-at-Risk
DC	Direct Current
DCPF	DC Power Flow
HiGHS	High-performance Interior-point and Simplex solver
HVDC	High-Voltage Direct Current
JIT	Just-In-Time (compilation)
LACPF	Linearized AC Power Flow
LMP	Locational Marginal Price
LOPF	Linear Optimal Power Flow
LP	Linear Program
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
MAPE	Mean Absolute Percentage Error
MILP	Mixed-Integer Linear Program
MIP	Mixed-Integer Program
MP-LOPF	Multi-Period Linear Optimal Power Flow
MVA	Megavolt-Ampere
MW	Megawatt
MWh	Megawatt-hour
NLP	Nonlinear Program
OPF	Optimal Power Flow
p.u.	Per Unit
PyPSA	Python for Power System Analysis
RMSE	Root Mean Square Error
SAA	Sample Average Approximation
SoC	State of Charge
TTFX	Time To First eXecution

Introduction

Relevance of the Research

The transition of electric power systems toward low-carbon operation has turned power system simulation and electricity market optimization into a computational bottleneck rather than a modeling formality. The large-scale integration of variable renewable generation, energy storage, and sector coupling has multiplied the number of decision variables, time steps, and scenarios that a single study must evaluate. Planning a decarbonization pathway for a national grid now routinely requires solving optimization problems with millions of variables across thousands of representative hours, and repeating that solve for dozens of policy or weather scenarios.

In the academic and open-research community, PyPSA (Python for Power System Analysis) has become the de facto standard framework for this class of problems. It underpins flagship models such as PyPSA-Eur and PyPSA-Earth and supports a large body of peer-reviewed work on renewable energy systems. Its strength is a clean, high-level modeling layer built on the Python scientific stack. That same layer, however, is also the source of a practical limitation: for large networks and iterative scenario studies, the time spent building the optimization model in Python — assembling constraints through pandas and the algebraic-modeling layer — can dominate the total runtime and reach hours per solve, well before the numerical solver itself begins work.

The Julia programming language offers a direct way to address this layer. Julia combines a high-level, dynamically typed syntax with just-in-time compilation to native code, and its JuMP modeling framework compiles the constraint-assembly step instead of interpreting it. This makes Julia a credible candidate for reimplementing the modeling layer that limits PyPSA, while delegating the ac-

tual numerical solving to the same industrial solvers (HiGHS, Ipopt) that PyPSA already uses.

At the same time, the operational use of these models increasingly depends on forecasts of uncertain quantities — most importantly electricity demand. A dispatch schedule that is optimal for a single point forecast can be far from optimal, or even infeasible, once the realized demand deviates from that forecast. This motivates coupling data-driven forecasting with the physics-based optimization, so that the optimizer is informed not only by a predicted demand profile but also by a calibrated description of its uncertainty. The combination of a high-performance Julia solver layer with an uncertainty-aware machine-learning forecast defines the "AI-assisted" framing of this work and explains its relevance to both the power systems and the scientific computing communities.

Problem Statement

Despite Julia's well-documented performance characteristics, no actively maintained, self-contained reimplementation of PyPSA's core computational modules currently exists, and the question of how much of PyPSA's runtime is actually recoverable by changing the modeling layer has not been answered with reproducible measurements. Equally, the integration of a statistically validated demand forecast into a stochastic dispatch formulation within a single native framework remains largely unexplored. The present work is organized around the following concrete questions:

- Can PyPSA's core algorithms — DC and AC power flow, linear optimal power flow, and unit commitment — be reimplemented in Julia while preserving numerical equivalence with the Python reference?
- What speedup is actually attributable to the Julia modeling layer, once both implementations are made to call the same underlying solver, and how does that speedup scale with network size and planning horizon?
- Can a neural load forecaster be equipped with distribution-free prediction intervals that provide a guaranteed coverage level suitable for dispatch optimization?

- Does propagating forecast uncertainty into a scenario-based stochastic dispatch yield decision-relevant risk measures that a deterministic formulation cannot provide?

Object and Subject of Research

The object of research is the set of computational methods used for steady-state power system simulation and for the optimization of electricity market dispatch, together with the data-driven forecasting methods that supply their demand inputs.

The subject of research is the implementation of these methods in the Julia programming language with a PyPSA-compatible interface, the quantitative comparison of their performance against the PyPSA reference implementation, and the integration of an uncertainty-aware load forecast into a stochastic dispatch formulation within that implementation.

Purpose of the Research

The purpose of this thesis is to design, implement, and validate a self-contained Julia framework that reproduces the core simulation and optimization capabilities of PyPSA with measurable performance gains, and to extend it with an AI-assisted layer that forecasts electricity demand under uncertainty and propagates that uncertainty into the dispatch optimization.

Research Tasks

To achieve this purpose, the following tasks are addressed:

1. Implement and validate DC and AC power flow solvers in Julia, establishing numerical equivalence with PyPSA on standard test networks.
2. Implement single-period and multi-period Linear Optimal Power Flow (LOPF) with energy storage and variable renewable generation, including the extraction of locational marginal prices from the dual solution.

3. Extend the LOPF to a Unit Commitment formulation as a mixed-integer linear program with minimum up/down times and start-up costs.

4. Design a reproducible benchmarking methodology and quantify the speedup of the Julia implementation over PyPSA across all solver types, a range of network sizes, and several planning horizons.

5. Develop a machine-learning load forecaster, equip it with conformal prediction intervals that guarantee a prescribed marginal coverage, and integrate the resulting demand scenarios into a sample-average-approximation stochastic LOPF.

Scientific Novelty

The scientific novelty of the work consists in the following:

- A self-contained, validated Julia reimplementation of PyPSA's core computational modules with a PyPSA-compatible construction API, requiring no Python dependency at solve time, in contrast to prior partial ports that retained Python in the workflow.

- A reproducible, multi-seed decomposition of the Julia-versus-PyPSA performance difference that attributes the speedup to the model-assembly layer rather than to the numerical solver, reported as compute-bound figures with means and standard deviations rather than headline ratios.

- The integration, within a single native framework, of a conformal-prediction-calibrated demand forecast with a scenario-based stochastic dispatch, producing distribution-free coverage guarantees and decision-relevant risk measures (expected cost and conditional value-at-risk) that are unavailable to deterministic dispatch.

Practical Significance

The resulting `PowerFlowJulia` framework provides a drop-in path for PyPSA users to accelerate the model-building stage of large or iterative studies

without changing their numerical backend, and demonstrates a concrete pipeline in which a calibrated forecast informs risk-aware dispatch decisions. The implementation, benchmark scripts, and data pipeline are released as a reproducible package.

Thesis Structure

The thesis follows a survey–design–implement–evaluate arc. Chapter 1 reviews the state of the art: the PyPSA framework and its modeling layer, the existing Julia power-system ecosystem, and the relevant literature on load forecasting and stochastic dispatch, and it identifies the research gap addressed here. Chapter 2 develops the mathematical and methodological foundations of every implemented method — the DC and AC power flow models, the LOPF and unit commitment formulations, the load-forecasting model with conformal prediction, and the stochastic LOPF — together with the benchmarking protocol. Chapter 3 presents the Julia implementation of each module, validates it against PyPSA, and reports the performance benchmarks and the results of the AI-assisted dispatch experiments. Chapter 5 synthesizes the findings, states the conclusions regarding Julia's suitability as a high-performance substitute for PyPSA and the value of AI-assisted dispatch, and outlines directions for future work.

Chapter 1

Literature Review

This chapter surveys the body of work on which the thesis builds. It first examines PyPSA and the structure of its modeling layer, which is the reference framework and the source of the performance limitation under study. It then reviews the Julia scientific-computing and mathematical-optimization ecosystem and the existing Julia tools for power-system analysis, distinguishing what each provides from what this thesis sets out to deliver. The final sections cover the data-driven components: short-term load forecasting, conformal prediction for distribution-free uncertainty quantification, and stochastic optimization for power systems. The chapter closes by stating the research gap that motivates the work.

1.1 Power System Analysis and the PyPSA Framework

Optimal power flow and economic dispatch are long-standing problems in power systems engineering, with formulations that date back several decades [18], [31]. The DC power flow approximation, in which the nonlinear AC equations are linearized around a flat voltage profile, has been the workhorse for market and planning studies since its formalization [28], because it reduces the network model to a sparse linear system while retaining the active-power flow behavior that dominates dispatch economics. The locational marginal price (LMP), which assigns a distinct energy price to each network node according to the dual of the nodal balance constraint, formalizes the economic interpretation of congestion and underpins modern electricity markets [21], [26].

PyPSA (Python for Power System Analysis) [4] has become the dominant open-source framework for this class of problems in the research community. It provides a high-level, component-oriented data model — buses, lines, generators, loads, storage units, and links — and assembles linear and mixed-integer optimization problems over user-defined time series. PyPSA is the engine behind continental-scale models such as PyPSA-Eur [13] and the global PyPSA-Earth [23], and it has supported a large volume of published energy-system research.

The framework's design also exposes a performance trade-off relevant to this thesis. PyPSA constructs its optimization problems in Python, building constraints through the pandas data-handling stack and the linopy algebraic-modeling layer [10] before handing the assembled problem to a numerical solver such as HiGHS [14] for linear and mixed-integer programs, or Ipopt [30] for nonlinear ones. For small problems this construction cost is negligible, but as the number of buses, snapshots, and scenarios grows, the time spent assembling the model in interpreted Python can come to dominate the wall-clock runtime, independently of the solver. This observation — that the modeling layer, not the solver, is the recoverable cost — frames the performance question of the present work.

1.2 Julia for Scientific Computing and Optimization

Julia [2] was designed to resolve the "two-language problem", in which a high-level language is used for prototyping and a low-level one for performance. Through type inference, multiple dispatch, and just-in-time compilation to native code via LLVM, Julia attains performance close to C while retaining a dynamic, high-level syntax. For the present work, the decisive feature is that performance-critical inner loops — such as the assembly of an optimization model from a component list — compile to specialized native code rather than being interpreted.

The JuMP modeling framework [8], [20] is the central piece of the Julia optimization ecosystem. JuMP provides an algebraic modeling language embedded in Julia and a solver-independent interface to a wide range of linear, mixed-integer, and nonlinear solvers, including the same HiGHS and Ipopt backends that PyPSA uses. Because JuMP compiles the model-building step, it has been reported to assemble large optimization problems substantially faster than interpreted Python modeling layers, which is precisely the cost component that limits PyPSA at scale.

JuMP therefore makes it possible to reimplement PyPSA's modeling layer in a compiled setting while keeping the numerical solver — and hence the achieved optimum — identical, isolating the language-level contribution to performance.

1.3 Existing Julia Tools for Power System Analysis

Several projects have addressed power-system computation in Julia, but none combines an actively maintained, self-contained implementation with a PyPSA-compatible interface. This section reviews the most relevant ones.

PSA.jl [25] is the closest existing attempt at a direct PyPSA port to Julia. Developed as a companion to PyPSA, it implements a subset of the LOPF functionality using JuMP and HiGHS. However, PSA.jl is not self-contained: the network must first be constructed in Python with PyPSA, serialized to CSV files, and only then imported into Julia for solving. This workflow preserves Python as an unavoidable dependency and therefore forfeits a substantial part of the potential performance benefit. The project has moreover been unmaintained since early 2021 and is incompatible with current Julia and JuMP releases, which renders it practically unusable for new work.

EnergyModels.jl [12] was conceived as a more ambitious successor that would rewrite PyPSA's full optimization core natively in Julia, motivated by the same performance arguments pursued here. The project was nevertheless abandoned around 2019 without reaching a usable state, and no active development has followed.

PowerModels.jl [7], developed at Los Alamos National Laboratory, is a mature and actively maintained Julia package for steady-state power-network optimization. It provides a rigorous framework for AC and DC optimal power flow formulations and supports many solvers through JuMP. PowerModels.jl is the most technically complete tool in the Julia power-systems ecosystem, and the present thesis uses it as the solver backend for the full AC power flow module. It does not, however, replicate PyPSA's higher-level abstractions: it offers no direct equivalent of PyPSA's economic-dispatch interface, multi-carrier energy model, time-series optimization, storage dispatch, or capacity planning.

PowerSimulations.jl [19], developed at the National Renewable Energy Laboratory, provides production-cost modeling and economic dispatch in Julia on

top of the PowerSystems.jl data model. It is actively maintained and covers part of the optimization functionality present in PyPSA. It constitutes, however, an independent architecture rather than a PyPSA port: its component abstractions, data model, and API differ substantially from PyPSA's, so it cannot serve as a drop-in replacement for existing PyPSA-based workflows.

Table 1.1: Comparison of Julia-based power system frameworks with respect to this thesis

Project	**Status**	**PyPSA API**	**Self-contained**	**Multi-period**
PSA.jl	Abandoned (2021)	Partial	No	No
EnergyModels.jl	Abandoned (2019)	Partial	No	No
PowerModels.jl	Active	No	Yes	No
PowerSimulations.jl	Active	No	Yes	Yes
This thesis	**Active**	**Partial**	**Yes**	**Partial**

The survey shows that the two projects that most directly attempted a PyPSA port (PSA.jl, EnergyModels.jl) have been abandoned for several years, while the two active alternatives (PowerModels.jl, PowerSimulations.jl) follow independent architectures and offer no PyPSA-compatible interface. No actively maintained, self-contained Julia reimplementation of PyPSA's core algorithms currently exists.

1.4 Short-Term Load Forecasting

Short-term load forecasting is a mature field with a literature spanning statistical, machine-learning, and, more recently, probabilistic approaches [11]. Classical baselines remain strong and are the honest reference against which any learned model must be judged. Two are used throughout this thesis: the persistence forecast, which predicts that tomorrow equals today, and the seasonal-naive forecast, which predicts that a given hour equals the same hour one week earlier, thereby capturing the dominant weekly cycle of electricity demand at no modeling cost.

Among learned models, recurrent neural networks — and in particular the Long Short-Term Memory (LSTM) architecture [9] — are widely applied to load forecasting because their gated memory can in principle capture the daily and weekly periodicities of demand. Other model families are also common; gradient-boosted decision trees, popularized by the XGBoost implementation [6], are a

strong baseline for tabular forecasting on engineered calendar and lag features and are noted here as an alternative direction. This thesis adopts an LSTM as its forecaster and evaluates it against the naive baselines. The probabilistic-forecasting literature further argues that, for operational decision-making, a calibrated description of forecast uncertainty is often more valuable than a marginal improvement in point accuracy [11], a position that directly motivates the uncertainty-quantification component below.

1.5 Conformal Prediction

Conformal prediction provides prediction intervals with a finite-sample, distribution-free coverage guarantee: under the sole assumption that the data are exchangeable, the constructed interval contains the true value with at least a user-specified probability, regardless of the underlying forecasting model or the data distribution [1], [29]. The split (inductive) variant computes non-conformity scores on a held-out calibration set and uses an appropriate empirical quantile of those scores to size the interval. This model-agnostic property is attractive for the present work because it allows the same calibration procedure to be applied to the forecaster, yielding guaranteed-coverage intervals that can be propagated into the dispatch optimization. It also separates the concern of point accuracy from the concern of coverage, which is consistent with the probabilistic-forecasting argument made above.

1.6 Stochastic Optimization in Power Systems

When demand is uncertain, a dispatch optimized for a single point forecast is generally suboptimal once the realized demand deviates from it. Stochastic programming addresses this by optimizing over a set of scenarios rather than a single realization. The Sample Average Approximation (SAA) method [27] approximates the expected-cost objective by an empirical average over a finite scenario sample, with the approximation improving as the number of scenarios grows; it is a standard and theoretically grounded approach for two-stage stochastic problems such as scenario-based dispatch.

Optimizing expected cost alone, however, ignores tail risk. The Conditional

Value-at-Risk (CVaR) [24] measures the expected cost in the worst fraction of scenarios and is a coherent risk measure with a convex, optimization-friendly representation. Reporting CVaR alongside the expected cost characterizes both the average and the adverse-case behavior of a dispatch schedule under demand uncertainty — information that a deterministic formulation cannot provide. This thesis combines conformal-calibrated demand scenarios with an SAA stochastic LOPF and reports both expected cost and CVaR.

1.7 Research Gap

The reviewed literature reveals a two-part gap. First, on the computational side, no actively maintained, self-contained Julia reimplementation of PyPSA's core modules exists: prior ports either retained Python in the workflow or were abandoned, and the active Julia tools follow non-PyPSA architectures. The extent to which PyPSA's runtime is recoverable by replacing only the modeling layer — while keeping the numerical solver fixed — has not been quantified with a reproducible, multi-seed protocol. Second, on the data-driven side, while load forecasting, conformal prediction, and stochastic dispatch are individually well studied, their integration into a single native framework — in which a conformally calibrated forecast supplies the scenarios for a risk-aware stochastic dispatch — remains largely unaddressed. The present thesis targets both parts of this gap: a validated, self-contained, PyPSA-compatible Julia implementation with a reproducible performance comparison, extended by an AI-assisted layer that couples calibrated forecasting with stochastic, risk-aware optimization.

Chapter 2

Design and Methodology

This chapter develops the theoretical and methodological foundations on which the remainder of the thesis rests. The first three sections derive the mathematical model underlying each algorithm: the DC power flow as a sparse linear system obtained by linearising the full AC equations, the AC power flow as a system of nonlinear nodal balance equations solved by Newton–Raphson iteration, and the Linear Optimal Power Flow as a linear programme that couples generator dispatch decisions with DC network flow constraints. The subsequent sections survey the Julia and Python software ecosystems used for implementation and highlight the key architectural differences between them. The chapter concludes by characterising the randomised network generator constructed to produce reproducible test cases of controlled size, and by specifying the JIT-aware timing methodology that ensures statistically valid performance comparisons between the two language implementations.

2.1 DC Power Flow: Mathematical Foundation

2.1.1 Power System Model

A power network is modeled as a graph $G = (\mathcal{N}, \mathcal{E})$, where $\mathcal{N}$ is the set of n buses (nodes) and $\mathcal{E}$ is the set of transmission lines (edges). Each bus i is characterized by its voltage V_i and phase angle θ_i. Each line $(i, j) \in \mathcal{E}$ has reactance x_{ij} and susceptance $b_{ij} = 1/x_{ij}$ (in per-unit) or $b_{ij} = V_{\text{nom}}^2/x_{ij}$ (in MW/rad when using physical units).

2.1.2 Susceptance Matrix Construction

The DC power flow approximation assumes that: (1) all voltage magnitudes equal their nominal value ($|V_i| \approx 1$ p.u.), (2) line resistances are negligible ($r_{ij} \ll x_{ij}$), (3) angle differences between adjacent buses are small ($\sin\theta \approx \theta$).

Under these assumptions, the active power injected at bus i is:

$$P_i = \sum_{j \in \mathcal{N}} B_{ij}\, \theta_j, \tag{2.1}$$

where $\mathbf{B} \in \mathbb{R}^{n \times n}$ is the *susceptance matrix* (also called the DC admittance matrix or B-matrix):

$$B_{ij} = \begin{cases} \sum_{k \neq i} b_{ik} & \text{if } i = j \\ -b_{ij} & \text{if } (i,j) \in \mathcal{E} \\ 0 & \text{otherwise} \end{cases} \tag{2.2}$$

The diagonal entries B_{ii} accumulate the susceptances of all lines incident to bus i, while off-diagonal entries B_{ij} represent the negative susceptance of the line between buses i and j. The matrix $\mathbf{B}$ is symmetric, sparse, and singular (the vector $\mathbf{1}$ lies in its null space).

2.1.3 Slack Bus and Reduced System

Since the absolute voltage angle has no physical meaning, we designate bus 1 as the *slack bus* (or reference bus) with $\theta_1 = 0$. This removes the singularity by reducing the system: we partition buses into the slack bus (index 1) and the remaining $n-1$ buses (indices $2, \ldots, n$), and solve the reduced linear system:

$$\hat{\mathbf{B}}\, \hat{\boldsymbol{\theta}} = \hat{\mathbf{P}}, \tag{2.3}$$

where $\hat{\mathbf{B}} = \mathbf{B}[2:n,\, 2:n]$ is the reduced susceptance matrix (always non-singular for connected networks), $\hat{\boldsymbol{\theta}} = (\theta_2, \ldots, \theta_n)^\top$ is the vector of unknown angles, and $\hat{\mathbf{P}} = (P_2, \ldots, P_n)^\top$ is the vector of net power injections (generation minus load) at all non-slack buses.

2.1.4 Active Power Flows

Once $\boldsymbol{\theta}$ is obtained, the active power flow on line (i, j) from bus i to bus j is:

$$p_{ij} = b_{ij}\,(\theta_i - \theta_j). \tag{2.4}$$

2.1.5 Algorithm Design

The Julia implementation follows four steps:

1. Assemble the $n \times n$ susceptance matrix $\mathbf{B}$ from the line list.
2. Compute the net injection vector $\mathbf{P} = \mathbf{P}^{\text{gen}} - \mathbf{P}^{\text{load}}$.
3. Solve the reduced system (2.3) using Julia's built-in backslash operator (which dispatches to LAPACK for dense or SuiteSparse for sparse matrices).
4. Compute per-line flows via Equation (2.4).

For large networks (e.g. $n > 500$), the susceptance matrix is stored as a sparse matrix to exploit the sparsity of real transmission networks. Typical transmission grids have $|\mathcal{E}| \sim 1.5\,n$, resulting in a matrix density below 1%.

2.2 AC Power Flow: Mathematical Foundation

2.2.1 Full AC Power Flow Equations

The AC power flow model represents the full nonlinear relationships between complex voltages and power injections. At each bus i, the complex voltage is $V_i = |V_i|e^{j\theta_i}$. The complex power injection is $S_i = P_i + jQ_i$, where P_i is active power and Q_i is reactive power.

The nodal power balance equations are derived from the admittance matrix $\mathbf{Y}_{\text{bus}}$:

$$P_i = |V_i| \sum_{j=1}^{n} |V_j|\,(G_{ij}\cos\theta_{ij} + B_{ij}\sin\theta_{ij})\,, \tag{2.5}$$

$$Q_i = |V_i| \sum_{j=1}^{n} |V_j|\,(G_{ij}\sin\theta_{ij} - B_{ij}\cos\theta_{ij})\,, \tag{2.6}$$

where $\theta_{ij} = \theta_i - \theta_j$, and $G_{ij} + jB_{ij}$ is the (i, j) entry of $\mathbf{Y}_{\text{bus}}$.

2.2.2 Newton-Raphson Solution Method

The AC power flow is solved iteratively using the Newton-Raphson method. Define the mismatch vector:

$$\mathbf{f}(\mathbf{x}) = \begin{pmatrix} \mathbf{P}^{\text{spec}} - \mathbf{P}(\mathbf{x}) \\ \mathbf{Q}^{\text{spec}} - \mathbf{Q}(\mathbf{x}) \end{pmatrix} = \mathbf{0}, \tag{2.7}$$

where $\mathbf{x} = (\boldsymbol{\theta}^\top, |\mathbf{V}|^\top)^\top$ is the state vector. The Newton-Raphson update at iteration k is:

$$\mathbf{J}^{(k)}\, \Delta\mathbf{x}^{(k)} = \mathbf{f}(\mathbf{x}^{(k)}), \qquad \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \Delta\mathbf{x}^{(k)}, \tag{2.8}$$

where $\mathbf{J} = \partial\mathbf{f}/\partial\mathbf{x}$ is the Jacobian matrix, which has the block structure:

$$\mathbf{J} = \begin{pmatrix} \partial\mathbf{P}/\partial\boldsymbol{\theta} & \partial\mathbf{P}/\partial|\mathbf{V}| \\ \partial\mathbf{Q}/\partial\boldsymbol{\theta} & \partial\mathbf{Q}/\partial|\mathbf{V}| \end{pmatrix}. \tag{2.9}$$

Convergence is declared when $\|\mathbf{f}(\mathbf{x}^{(k)})\|_\infty < \varepsilon$ (typically $\varepsilon = 10^{-6}$ p.u.).

2.2.3 Per-Unit System

AC power flow computations use the *per-unit* (p.u.) system to normalize quantities and improve numerical conditioning. For a system with base apparent power $S_{\text{base}} = 100\,\text{MVA}$ and base voltage V_{base}:

$$P_{\text{p.u.}} = \frac{P_{\text{MW}}}{S_{\text{base}}}, \quad Z_{\text{p.u.}} = \frac{Z_\Omega \cdot S_{\text{base}}}{V_{\text{base}}^2}. \tag{2.10}$$

All generator outputs, loads, and line impedances must be converted to per-unit before solving. The Julia implementation performs this conversion explicitly, a step that was a key source of errors during development.

2.2.4 PowerModels.jl Integration

For the AC power flow implementation, this thesis uses **PowerModels.jl** [7], a Julia package that provides a general framework for formulating and solving

power network optimization problems. PowerModels.jl uses a standardized data dictionary format (inspired by Matpower [32]) and interfaces with Ipopt [30] for nonlinear optimization.

The AC power flow is cast as a nonlinear optimization problem (minimizing generation cost subject to power flow constraints), which Ipopt solves via interior-point methods.

2.3 Linearized AC Power Flow: Mathematical Foundation

2.3.1 Motivation and Position in the Hierarchy

The DC power flow (Section 2.1) and full AC power flow (Section 2.2) represent two extremes: the DC approximation is computationally trivial but ignores resistance, reactive power, and voltage magnitude variations; the full Newton–Raphson AC solution is exact but requires iterative nonlinear solves. The *Linearized AC Power Flow* (LACPF) occupies the middle ground: it retains the computational simplicity of a single linear solve while recovering both voltage magnitude deviations and reactive power flows that the DC approximation discards.

2.3.2 First-Order Taylor Linearisation

The LACPF is derived by applying a first-order Taylor expansion to the full AC power flow equations around the *flat start* operating point, where all voltage magnitudes equal $1\,\text{p.u.}$ and all angles equal zero. Let $\Delta\boldsymbol{\theta} = \boldsymbol{\theta} - \mathbf{0}$ and $\Delta|\mathbf{V}| = |\mathbf{V}| - \mathbf{1}$ denote deviations from the flat start. The linearised nodal power balance is:

$$\begin{pmatrix} \mathbf{B}' & \mathbf{G}' \\ -\mathbf{G}' & -\mathbf{B}' \end{pmatrix} \begin{pmatrix} \Delta\boldsymbol{\theta} \\ \Delta|\mathbf{V}| \end{pmatrix} = \begin{pmatrix} \mathbf{P} \\ \mathbf{Q} \end{pmatrix}, \tag{2.11}$$

where $\mathbf{B}' \in \mathbb{R}^{n\times n}$ and $\mathbf{G}' \in \mathbb{R}^{n\times n}$ are the *susceptance Laplacian* and *conductance Laplacian* of the network graph, respectively:

$$B'_{ij} = \begin{cases} \sum_{k\neq i} b_{ik} & i = j \\ -b_{ij} & i \neq j \end{cases}, \qquad G'_{ij} = \begin{cases} \sum_{k\neq i} g_{ik} & i = j \\ -g_{ij} & i \neq j \end{cases}, \tag{2.12}$$

with branch susceptance $b_{ij} = x_{ij}/(r_{ij}^2 + x_{ij}^2)$ and conductance $g_{ij} = r_{ij}/(r_{ij}^2 + x_{ij}^2)$ in per-unit. The slack bus is removed by reducing the system to $2(n-1)\times 2(n-1)$, analogously to the DC power flow reduction.

2.3.3 Comparison with DC Power Flow

Setting $\mathbf{G}' = \mathbf{0}$ (lossless lines, $r = 0$) and ignoring the $\mathbf{Q}$ equation reduces Equation (2.11) to the standard DC power flow equation $\mathbf{B}'\Delta\boldsymbol{\theta} = \mathbf{P}$. The LACPF therefore strictly extends the DC approximation by:

1. Including line resistance through $\mathbf{G}'$, improving active power flow accuracy.
2. Solving for voltage magnitude deviations $\Delta|\mathbf{V}|$ that the DC model cannot provide.
3. Providing first-order estimates of reactive power flows $\mathbf{Q}$.

However, reactive power estimates from the LACPF are known to have larger errors than active power estimates, because reactive power is more sensitive to second-order voltage effects that are discarded in the linearisation [28].

2.4 Linear Optimal Power Flow: Mathematical Foundation

2.4.1 Problem Formulation

The Linear Optimal Power Flow (LOPF) combines the DC power flow approximation with economic dispatch optimization. The goal is to dispatch generators to minimize total generation cost while satisfying network constraints. The LOPF is a linear program (LP):

$$\min_{\mathbf{P}^{\text{gen}},\,\boldsymbol{\theta}} \quad \sum_{i\in\mathcal{G}} c_i\, P_i^{\text{gen}} \tag{2.13}$$

subject to:

$$\theta_{\text{ref}} = 0 \tag{2.14}$$

$$\sum_{j\in\mathcal{N}} B_{ij}\,\theta_j = P_i^{\text{gen}} - P_i^{\text{load}}, \quad \forall i \in \mathcal{N} \tag{2.15}$$

$$-F_{ij}^{\max} \le b_{ij}\,(\theta_i - \theta_j) \le F_{ij}^{\max}, \quad \forall (i,j) \in \mathcal{E} \tag{2.16}$$

$$0 \le P_i^{\text{gen}} \le P_i^{\max}, \quad \forall i \in \mathcal{G} \tag{2.17}$$

where $\mathcal{G} \subseteq \mathcal{N}$ is the set of generator buses, c_i is the marginal cost of generator i (€/MWh), $F_{ij}^{\max}$ is the thermal limit of line (i,j), and $P_i^{\max}$ is the installed generator capacity.

2.4.2 Connection to DC Power Flow

Equation (2.15) is precisely the DC power flow balance (2.1), with the generator output P_i^{gen} as an additional decision variable. The LOPF thus extends DC power flow by making generator dispatch optimal rather than fixed. The voltage angles $\boldsymbol{\theta}$ serve as intermediate variables linking generation and line flows.

2.4.3 LP Solver Interface Design via JuMP

The Julia implementation uses **JuMP.jl** [20], a domain-specific modeling language embedded in Julia, to declare the optimization model. JuMP separates the model description from the solver, enabling transparent solver switching. The workflow is:

1. Declare a `Model` object with the HiGHS solver backend.
2. Register decision variables (`@variable`) with bounds.
3. Add constraints (`@constraint`) for power balance and line limits.
4. Set objective (`@objective`) to minimize marginal cost.

5. Call `optimize!(model)` and extract results.

HiGHS [14] is used as the LP solver for both the Julia (via HiGHS.jl) and Python (via PyPSA/linopy) implementations, ensuring a fair comparison of language overhead rather than solver differences.

2.5 Technology Stack and Tool Selection

2.5.1 Julia Ecosystem

The Julia implementation uses the following packages, all available through Julia's built-in package manager (`Pkg`):

Table 2.1: Julia packages used in the implementation

Package	Version	Purpose
LinearAlgebra	stdlib	Dense matrix operations, backslash solver
SparseArrays	stdlib	Sparse matrix construction and operations
JuMP	1.29	Optimization modeling language
HiGHS	1.21	LP/MIP solver
PowerModels	0.21	AC power flow, network data format
Ipopt	1.14	Nonlinear interior-point solver

2.5.2 Python Ecosystem

The Python reference implementation uses **PyPSA** [4], which internally relies on **linopy** for LP/MILP modeling and **HiGHS** for LP solving.

Table 2.2: Python packages used as reference implementation

Package	Version	Purpose
PyPSA	0.28+	Power system analysis framework
NumPy	1.26+	Array operations
linopy	0.3+	LP modeling (used internally by PyPSA)
HiGHS	1.7+	LP solver backend

2.5.3 Comparative Analysis of Ecosystems

Table 2.3 summarizes the key differences between the Julia and Python ecosystems relevant to this thesis.

Table 2.3: Comparison of Julia and Python ecosystems for power system computation

Aspect	**Julia**	**Python**
Execution model	JIT-compiled (LLVM)	Interpreted
Linear algebra	Direct LAPACK calls	NumPy (LAPACK + overhead)
LP modeling	JuMP (native)	linopy/Pyomo (Python objects)
Startup time	High (TTFX)	Low
Matrix operations	Native performance	C extension calls
Type system	Multiple dispatch	Dynamic typing
Sparse matrices	Native (stdlib)	scipy.sparse

A critical difference is Julia's *time to first execution* (TTFX): the first call to any function triggers JIT compilation, which can take several seconds. However, subsequent calls run at native speed. The benchmarks in this thesis exclude warmup time to measure steady-state performance.

2.6 Benchmark Methodology

2.6.1 Random Network Generator

To enable reproducible, fair comparisons between Julia and Python implementations, a random network generator was implemented using the same algorithm and the same random seed (`seed=42`) in both languages; synthetic networks of controlled size are an established tool for studying power-system algorithms at scale [3]. The generator produces a connected network by:

1. Constructing a *spanning tree*: bus i connects to bus $i + 1$ with reactance $x \sim \mathcal{U}(0.05,\ 0.50)$ p.u., ensuring connectivity.

2. Adding approximately $\lfloor n/3 \rfloor$ additional random edges to create mesh topology.

3. Assigning random loads $P^{\text{load}} \sim \mathcal{U}(50, 500)$ MW to 70% of non-slack buses.

4. Placing generators at bus 1 (slack, capacity = $1.1 \times \sum P^{\text{load}}$) and at every 4th bus.

Networks are generated for sizes $n \in \{3, 10, 50, 100, 500, 1000, 2000\}$ buses for DC power flow, and $n \in \{3, 10, 50, 100, 500\}$ buses for LOPF (larger sizes were omitted due to Python runtime constraints).

2.6.2 Timing Protocol

A single fair protocol is applied to both implementations so that the comparison isolates the language-level contribution to performance:

1. **Warm-up.** Julia's JIT compiler compiles on first call, so each measurement is preceded by one untimed warm-up call that excludes compilation; steady-state performance is what is reported.

2. **Build once, time the solve.** For every scenario the network is constructed once; only the per-call operation (optimisation-model assembly plus solve) is timed, on both sides. This excludes one-time network construction from both, and isolates the model-assembly cost that the thesis studies. The Julia side uses `BenchmarkTools.jl` (auto-tuned repeated samples); the Python side uses repeated `perf_counter` samples.

3. **Report the minimum.** The reported figure is the *minimum* over the samples. The minimum is the standard noise-robust estimator of intrinsic compute time, because operating-system jitter and garbage-collection pauses can only add time, never subtract it; the mean (which the minimum complements) is inflated by such transients.

4. **Pin threads.** The HiGHS solver and the BLAS linear-algebra backend are pinned to a single thread on both sides, so neither implementation gains an unfair advantage from differing default thread counts.

For synthetic networks the same random generator and seeds are used in both languages; in addition, standard published networks (IEEE 118/300 and PEGASE 1354/2869) are loaded from MATPOWER files and benchmarked through the public API, giving a realistic-topology comparison alongside the controlled synthetic scaling.

2.6.3 Hardware and Software Environment

All benchmarks were executed on a single machine under identical conditions. The operating system is Windows 11 Pro (build 26100). Julia version 1.12 and Python 3.11 were used. No parallel computation or GPU acceleration was employed; all computations are single-threaded to ensure a direct comparison of sequential algorithmic performance.

2.7 Unit Commitment: Mathematical Foundation

Unit Commitment (UC) extends the multi-period LOPF by adding binary on/off decisions for generators, minimum operating time constraints, and fixed startup and shutdown costs. The resulting problem is a Mixed-Integer Linear Program (MILP); the formulation here follows the standard efficient thermal unit commitment model [5]. For each committable generator g and time period $t \in \{1, \ldots, T\}$, three binary variables are defined: $u_{g,t} \in \{0, 1\}$ (commitment status), $\sigma^{+}_{g,t} \in \{0, 1\}$ (startup event), and $\sigma^{-}_{g,t} \in \{0, 1\}$ (shutdown event), linked by the transition constraint:

$$u_{g,t} - u_{g,t-1} = \sigma^{+}_{g,t} - \sigma^{-}_{g,t}, \quad \forall g, t. \tag{2.18}$$

The dispatch bounds couple binary and continuous variables:

$$p_g^{\min} u_{g,t} \;\leq\; P_{g,t} \;\leq\; p_g^{\max} u_{g,t}. \tag{2.19}$$

Minimum up-time (T_g^{up}) and down-time (T_g^{dn}) are enforced by the rolling-window constraints:

$$\sum_{\tau=t-T_g^{up}+1}^{t} \sigma_{g,\tau}^{+} \leq u_{g,t}, \quad \forall t \geq T_g^{up}, \tag{2.20}$$

$$\sum_{\tau=t-T_g^{dn}+1}^{t} \sigma_{g,\tau}^{-} \leq 1 - u_{g,t}, \quad \forall t \geq T_g^{dn}. \tag{2.21}$$

The extended objective minimises dispatch cost plus fixed startup and shutdown costs:

$$\min \sum_{g,t} \left[c_g P_{g,t} + C_g^{su}\, \sigma_{g,t}^{+} + C_g^{sd}\, \sigma_{g,t}^{-} \right] \tag{2.22}$$

subject to the power balance (3.10) and storage dynamics (3.7)–(3.8) from Section 2.4. Because UC contains integer variables, Locational Marginal Prices cannot be extracted directly from the MILP dual; instead, the binary variables are fixed at their optimal values and a continuous LP relaxation is re-solved to recover price-consistent LMPs [21].

2.8 AI Component: LSTM Load Forecasting

2.8.1 Motivation

Demand uncertainty is a fundamental challenge in power system operation. Even modest forecast errors — on the order of 5–15% Mean Absolute Percentage Error (MAPE) — propagate into suboptimal dispatch, unnecessary reserve activations, and higher operational costs. The AI component of this thesis introduces a data-driven LSTM forecaster that produces not only a point forecast of the next 24-hour load profile, but also statistically valid prediction intervals, which are then fed into a stochastic dispatch optimization.

2.8.2 LSTM Architecture

A Long Short-Term Memory (LSTM) network [9] is used for its ability to capture temporal dependencies in load time series. The cell state dynamics at each

time step t are:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1};\, \mathbf{x}_t] + \mathbf{b}_f) \qquad \text{(forget gate)} \tag{2.23}$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1};\, \mathbf{x}_t] + \mathbf{b}_i) \qquad \text{(input gate)} \tag{2.24}$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1};\, \mathbf{x}_t] + \mathbf{b}_c) \qquad \text{(candidate)} \tag{2.25}$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \qquad \text{(cell state)} \tag{2.26}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \qquad \text{(hidden state)} \tag{2.27}$$

where $\sigma(\cdot)$ is the sigmoid function and $\odot$ denotes element-wise multiplication. The hidden state $\mathbf{h}_t$ at the last input step is passed through a two-layer (Dense) head to produce a 24-hour output vector.

The architecture used in this thesis processes a *weekly* input window of 168 hours so that the network sees the full weekly demand cycle. Each hour carries a feature vector rather than a scalar: the normalised load together with calendar encodings (sine/cosine of hour-of-day, sine/cosine of day-of-week, and a weekend flag), and optionally an exogenous temperature, giving six or seven input features per step. The encoder is an LSTM with hidden size 64, followed by an MLP head producing the 24-hour horizon. Rather than predicting the load level directly, the model predicts the *residual* to the seasonal-naive baseline (the load one week earlier), so that the strong weekly periodicity is supplied for free and the network only learns the deviation from it. The model is implemented in **Flux.jl** [15] and trained with the Adam optimiser [17] (learning rate 10^{-3}) with early stopping on a held-out validation set.

2.8.3 Training Protocol

The input data is structured as a batched sliding-window dataset: each sample consists of a 168-hour multi-feature input window and the subsequent 24-hour load target, with the load z-score normalised. Training is performed over full sequences in mini-batches (the recurrent layer processes the whole window and the last timestep is taken), which both preserves the LSTM's state across the window and is fast enough to train on multi-year data. The dataset is split chronologically into training, validation, and calibration subsets; model parameters are restored from the epoch with lowest validation RMSE.

2.8.4 Conformal Prediction Intervals

Standard neural network confidence intervals are not statistically guaranteed. This thesis instead applies *split conformal prediction* [1] to produce intervals with a distribution-free coverage guarantee.

The non-conformity score for a calibration sample (x_i, y_i) is the maximum absolute prediction error over the 24-hour horizon:

$$s_i = \max_{j=1}^{24} |\hat{y}_{ij} - y_{ij}|, \quad i = 1, \ldots, n_{\text{cal}}. \tag{2.28}$$

The conformal quantile at coverage level $1 - \alpha$ is:

$$\hat{q} = s_{\lceil (1-\alpha)(n_{\text{cal}}+1) \rceil}, \tag{2.29}$$

where scores are sorted in ascending order. The prediction interval for a new input x^* is:

$$[\hat{y}_j^* - \hat{q},\ \hat{y}_j^* + \hat{q}], \quad j = 1, \ldots, 24. \tag{2.30}$$

By Theorem 1 of [1], this interval satisfies $\mathbb{P}(y_j^* \in [\hat{y}_j^* - \hat{q}, \hat{y}_j^* + \hat{q}]) \geq 1 - \alpha$ for exchangeable data, without any distributional assumptions on the residuals.

2.9 Stochastic LOPF via Sample Average Approximation

2.9.1 Uncertainty Model

The conformal prediction intervals define a set of plausible 24-hour load profiles. To quantify the resulting cost uncertainty, S scenario profiles are sampled from $\mathcal{N}(\hat{\mathbf{y}}^*, \sigma_j^2)$ clipped to $[\hat{y}_j^* - \hat{q},\ \hat{y}_j^* + \hat{q}]$, where $\sigma_j = (\hat{q})/2$ is chosen so that approximately 95% of the Gaussian mass lies within the conformal interval.

2.9.2 Sample Average Approximation

The Sample Average Approximation (SAA) [27] replaces the true expectation $\mathbb{E}[f(x, \xi)]$ with an empirical average:

$$\min_{x} \frac{1}{S} \sum_{s=1}^{S} f(x, \xi_s), \tag{2.31}$$

where ξ_s is the s-th load scenario vector. In this thesis, the SAA is implemented by solving S independent multi-period LOPFs and aggregating the results:

$$\mathbb{E}[\text{cost}] \approx \frac{1}{S} \sum_{s=1}^{S} C^*(s), \qquad C^*(s) = \text{LOPF cost under scenario } s. \tag{2.32}$$

2.9.3 Conditional Value-at-Risk

Beyond the expected cost, the α-CVaR quantifies tail risk — the expected cost in the worst $(1 - \alpha)$ fraction of scenarios [24]:

$$\text{CVaR}_{1-\alpha} = \frac{1}{\lfloor (1-\alpha)S \rfloor} \sum_{s \in \mathcal{W}} C^*(s), \tag{2.33}$$

where $\mathcal{W}$ is the index set of the $\lfloor (1 - \alpha)S \rfloor$ highest-cost scenarios. In this thesis, $\alpha = 0.90$ (CVaR_{90}) is used, corresponding to the expected cost in the worst 10% of scenarios. CVaR is a coherent risk measure [24] and is preferred over Value-at-Risk because it captures the full tail distribution rather than a single quantile.

Chapter 3

Implementation and Experiments

This chapter presents the Julia implementations of the three power flow modules alongside their numerical validation and performance evaluation. The chapter opens with the definition of a canonical 3-bus test network that serves as a common validation fixture throughout. The DC power flow, AC power flow, and LOPF implementations are then described in turn: each section presents the core source code, discusses the engineering decisions and non-obvious challenges encountered during porting, and reports numerical results compared against the reference PyPSA implementation. The chapter concludes with a systematic performance benchmark spanning networks from 3 to 2,000 buses, followed by an analysis that accounts for the observed speedup patterns — including the opposing trends exhibited by DC power flow and LOPF as problem size grows.

3.1 Test Network Definition

All three implementations are validated on the same standard 3-bus test network, ensuring consistency across comparisons with PyPSA. The network is defined as follows:

- **Bus 1:** Generator G1, capacity $P_1^{\max} = 400\,\text{MW}$, marginal cost $c_1 = 20\,€/\text{MWh}$, reference bus ($\theta_1 = 0$).
- **Bus 2:** Generator G2, capacity $P_2^{\max} = 300\,\text{MW}$, marginal cost

$c_2 = 50\,€/\text{MWh}$; load $L_2 = 200\,\text{MW}$.

- **Bus 3:** Load $L_3 = 300\,\text{MW}$, no local generation.

All three transmission lines have reactance $x = 0.1\,\text{p.u.}$ (base: $S_{\text{base}} = 100\,\text{MVA}$, $V_{\text{base}} = 380\,\text{kV}$), giving a susceptance of:

$$b = \frac{S_{\text{base}}}{x} = \frac{100\,\text{MVA}}{0.1\,\text{p.u.}} = 1000\,\text{MW/rad}. \tag{3.1}$$

The susceptance matrix of this network is:

$$\mathbf{B} = \begin{pmatrix} 2000 & -1000 & -1000 \\ -1000 & 2000 & -1000 \\ -1000 & -1000 & 2000 \end{pmatrix}\ \text{MW/rad}. \tag{3.2}$$

3.2 DC Power Flow Implementation

3.2.1 Core Implementation

The DC power flow solver is implemented in `julia/dc_power_flow.jl`. The core function assembles the susceptance matrix and solves the linear system using Julia's backslash operator, which dispatches to LAPACK for dense matrices:

```
function dc_pf_solve(n_buses, lines, generators, loads; baseMVA
   =100.0)
    # Build susceptance matrix B (n x n)
    B = zeros(n_buses, n_buses)
    for (from, to, x) in lines
        b = baseMVA / x          # b [MW/rad] = S_base / x_pu
        B[from, from] += b
        B[to,   to  ] += b
        B[from, to  ] -= b
        B[to,   from] -= b
    end

    # Net injection vector P = P_gen - P_load
    P = zeros(n_buses)
    for (bus, p) in generators;  P[bus] += p;  end
    for (bus, p) in loads;       P[bus] -= p;  end

```

```
    # Remove slack bus (bus 1): solve reduced (n-1) x (n-1) system
    idx   = 2:n_buses
    theta = zeros(n_buses)
    theta[idx] = B[idx, idx] \ P[idx]    # Julia backslash -> LAPACK
    return theta
end
```

Listing 3.1: DC power flow: susceptance matrix and linear solve

3.2.2 Validation

For the DC power flow validation, the generator dispatch is fixed at $G_1 = 400\,\text{MW}$ (slack, absorbs imbalance), $G_2 = 100\,\text{MW}$ (the economically optimal dispatch from LOPF Scenario A). The net injections are:

$$\mathbf{P} = \begin{pmatrix} 400 \\ 100 - 200 \\ 0 - 300 \end{pmatrix} = \begin{pmatrix} 400 \\ -100 \\ -300 \end{pmatrix} \text{MW}. \tag{3.3}$$

Solving the reduced 2×2 system $\hat{\mathbf{B}}\hat{\boldsymbol{\theta}} = \hat{\mathbf{P}}$ analytically:

$$\begin{pmatrix} 2000 & -1000 \\ -1000 & 2000 \end{pmatrix} \begin{pmatrix} \theta_2 \\ \theta_3 \end{pmatrix} = \begin{pmatrix} -100 \\ -300 \end{pmatrix} \implies \begin{pmatrix} \theta_2 \\ \theta_3 \end{pmatrix} = \begin{pmatrix} -1/6 \\ -7/30 \end{pmatrix} \text{rad}. \tag{3.4}$$

Table 3.1 compares the Julia and PyPSA results on this network.

Table 3.1: DC power flow validation: Julia vs PyPSA (3-bus network, $G_1 = 400\,\text{MW}$, $G_2 = 100\,\text{MW}$). Branch flows agree exactly; voltage angles differ by a constant per-unit convention factor of $V_{\text{nom}}^2/S_{\text{base}} = 380^2/100 = 1444$ (see text).

Quantity	**Julia**	**PyPSA**	**Relationship**
θ_2 (rad)	-0.1667	-1.15×10^{-4}	ratio $= 1444$
θ_3 (rad)	-0.2333	-1.62×10^{-4}	ratio $= 1444$
p_{12} (MW)	166.7	166.7	$\Delta < 10^{-6}$
p_{13} (MW)	233.3	233.3	$\Delta < 10^{-6}$
p_{23} (MW)	66.7	66.7	$\Delta < 10^{-6}$

The branch flows — the physically and economically meaningful output of the DC power flow — agree to better than $10^{-6}\,\text{MW}$. The voltage angles differ by an exact constant factor of 1444 because the two implementations adopt different per-unit conventions for the line reactance. PyPSA interprets the `Line.x` attribute as an *ohmic* value and converts it internally to per-unit by dividing by $V_{\text{nom}}^2/S_{\text{base}} = 380^2/100 = 1444\,\Omega$, producing angles of order $10^{-4}\,\text{rad}$. The present implementation, following the MATPOWER convention, treats `x` as already in per-unit and forms the susceptance as $b = S_{\text{base}}/x$ in MW/rad, producing angles of order $10^{-1}\,\text{rad}$. Because the angle scale cancels exactly in the branch-flow expression $p_{km} = b_{km}(\theta_k - \theta_m)$, the resulting flows are identical. The difference is therefore a documented input-convention choice, not a modelling error; the same flows, dispatch, and prices are obtained under both conventions. Notably, this scale difference appears only in the synthetic toy network, where a bare per-unit reactance ($x = 0.1$) is supplied against a $380\,\text{kV}$ base; on the IEEE 14-bus case of Section 3.6, where the MATPOWER data carry physically consistent per-unit reactances, the Julia and PyPSA angles agree to better than 0.001° across all buses (Table 3.6). Figure 3.1 shows the voltage angles at each bus.

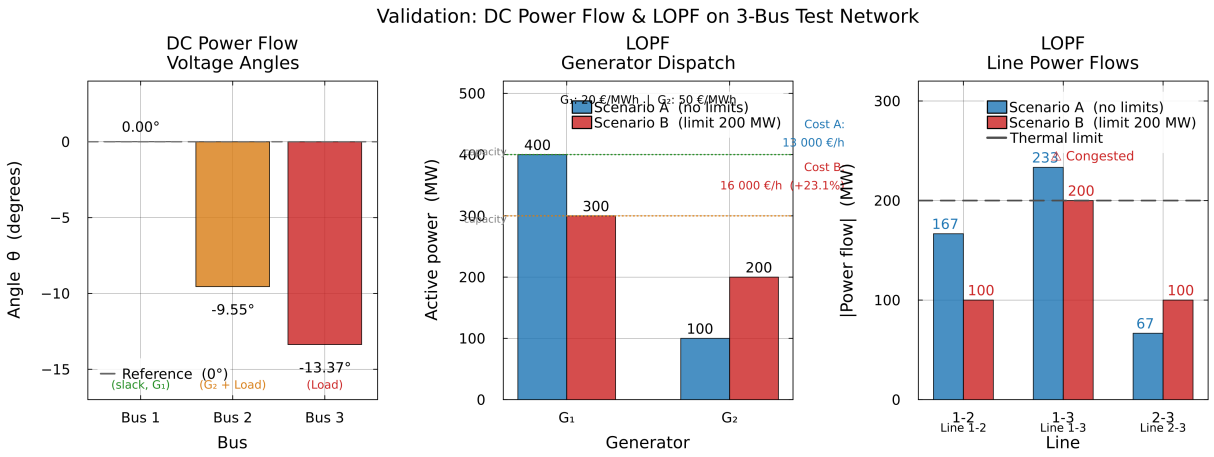

Fig. 3.1. Validation results on the 3-bus test network (left to right): (a) DC Power Flow voltage angles at each bus — Bus 1 is the reference ($\theta_1 = 0°$), with angles of $-9.55°$ and $-13.37°$ at Buses 2 and 3; (b) LOPF generator dispatch for Scenario A (unconstrained, blue) and Scenario B (200 MW thermal limit, red); (c) transmission line power flows for both scenarios, with the 200 MW thermal limit shown as a dashed line — Line 1-3 is at 116.7% loading in Scenario A and exactly at the limit in Scenario B.

3.3 AC Power Flow Implementation

3.3.1 PowerModels.jl Data Format

The AC power flow is implemented using PowerModels.jl, which requires a specific data dictionary format. Building this format from scratch revealed several non-obvious requirements:

1. **Per-unit conversion:** All quantities must be in p.u. ($S_{\text{base}} = 100\,\text{MVA}$). Generator output, load power, and line impedances must all be divided by their respective base values. Failing to convert leads to angles of hundreds or thousands of radians in the solver output.

2. **Mandatory shunt admittance fields:** Each branch requires explicit `g_fr`, `b_fr`, `g_to`, `b_to` fields (shunt conductance and susceptance at the from- and to-ends of the line). These are not auto-computed; for lossless lines, `g_fr = g_to = 0` and `b_fr = b_to =` $b_c/2$, where b_c is the line charging susceptance. This requirement was discovered by reading the PowerModels.jl source (`matpower.jl:454`).

3. **Angle bounds in radians:** The `angmin`/`angmax` fields must be in radians ($\pm\pi/3 \approx \pm 60°$). Supplying values in degrees makes the bounds effectively infinite.

4. **Branch flow extraction:** The `build_pf` formulation uses JuMP affine expressions (not variables) for branch flows, so they do not appear in the solution dictionary. Branch flows must be computed post-solution via the helper `calc_branch_flow_ac` from PowerModels.

3.3.2 Implementation

The key function converting the network to PowerModels format is shown below (condensed for clarity):

```
function network_to_powermodels(buses, lines, generators, loads,
                                v_nom=380.0; baseMVA=100.0)
    data = Dict{String,Any}(
        "baseMVA"  => baseMVA,
        "per_unit" => true,
        "shunt"    => Dict{String,Any}(),   # required empty
          dicts
        "dcline"   => Dict{String,Any}(),
        "storage"  => Dict{String,Any}(),
        "switch"   => Dict{String,Any}(), ...
    )
    # Branch: convert to per-unit + add shunt admittance fields
    Z_base = v_nom^2 / baseMVA          # base impedance [Ohm]
    br_r = r / Z_base;  br_x = x / Z_base
    br_b = 1.0 / sqrt(br_r^2 + br_x^2) * 0.1  # ~10% charging
    branch_data = Dict(
        "br_r"  => br_r,  "br_x"  => br_x,
        "g_fr"  => 0.0,   "b_fr"  => br_b/2.0,   # required!
        "g_to"  => 0.0,   "b_to"  => br_b/2.0,   # required!
        "angmin" => -pi/3, "angmax" => pi/3, ...)
    # Generator: per-unit active power
    gen_data = Dict("pg" => P_MW / baseMVA,
                    "pmax" => P_max / baseMVA, ...)
end
```

Listing 3.2: PowerModels.jl data format construction (key fields)

3.3.3 Branch Flow Extraction

After solving, branch flows are recovered in two steps: the solution is merged back into the data dictionary, and then the branch flows are computed from the recovered voltages:

```
PowerModels.update_data!(pm_data, result["solution"])
branch_flows = PowerModels.calc_branch_flow_ac(pm_data)
for (i, branch) in sort(branch_flows["branch"])
    pf_MW = branch["pf"] * baseMVA     # convert from p.u. to MW
    pt_MW = branch["pt"] * baseMVA
end
```

Listing 3.3: Extracting branch flows from PowerModels solution

3.3.4 Validation

The AC power flow implementation is validated on the same 3-bus test network defined in Section 3.1, with the generator dispatch fixed at $G_1 = 500\,\text{MW}$ (slack), loads $L_2 = 300\,\text{MW}$ and $L_3 = 200\,\text{MW}$, and line parameters $r = 0.01$ p.u., $x = 0.1$ p.u. ($S_{\text{base}} = 100\,\text{MVA}$, $V_{\text{base}} = 380\,\text{kV}$).

Table 3.2 compares the Julia (PowerModels.jl + Ipopt) and PyPSA (`network.pf()`) results.

Table 3.2: AC power flow validation: Julia vs PyPSA (3-bus network)

Quantity	**Julia**	**PyPSA**	**Max $\lvert\Delta\rvert$**
$\lvert V_1\rvert$ (p.u.)	1.000000	1.000000	0
$\lvert V_2\rvert$ (p.u.)	0.999982	0.999982	4.8×10^{-7}
$\lvert V_3\rvert$ (p.u.)	0.999984	0.999984	4.8×10^{-7}
θ_1 (rad)	0.000000	0.000000	0
θ_2 (rad)	-0.000185	-0.000185	4.1×10^{-7}
θ_3 (rad)	-0.000162	-0.000162	4.1×10^{-7}
p_{12} (MW)	266.67	266.67	3.5×10^{-3}
p_{13} (MW)	233.34	233.34	3.5×10^{-3}
p_{23} (MW)	-33.33	-33.33	3.5×10^{-3}

All quantities agree with PyPSA to better than 5×10^{-3} absolute error.

The small residual in line active power (3.5×10^{-3} MW) reflects the difference between Ipopt's interior-point convergence tolerance (10^{-8} p.u.) and PyPSA's Newton–Raphson tolerance (10^{-6} p.u.), not a modelling error. Voltage magnitudes and angles agree to $< 5 \times 10^{-7}$, confirming that the per-unit conversion, admittance matrix construction, and PowerModels.jl data format are all correct.

3.4 Linearized AC Power Flow Implementation

3.4.1 Implementation

The Linearized AC Power Flow is implemented in `julia/solvers/linear_ac_pf.j` as a direct linear solve of Equation (2.11). The key difference from the DC power flow is the construction of both the susceptance Laplacian $\mathbf{B}'$ and the conductance Laplacian $\mathbf{G}'$, and the assembly of the $2(n-1) \times 2(n-1)$ coupled system:

```
z_base = v_nom^2 / baseMVA           # 380^2 / 100 = 1444 Ohm
for (from, to, r, x) in lines
    r_pu = r / z_base;  x_pu = x / z_base
    denom = r_pu^2 + x_pu^2
    g_ij = r_pu / denom;  b_ij = x_pu / denom
    # Susceptance Laplacian B'
    B[from,from] += b_ij;  B[to,to] += b_ij
    B[from,to]   -= b_ij;  B[to,from] -= b_ij
    # Conductance Laplacian G'
    G[from,from] += g_ij;  G[to,to] += g_ij
    G[from,to]   -= g_ij;  G[to,from] -= g_ij
end
# 2(n-1) x 2(n-1) coupled system
A = [ B_r  G_r ; -G_r  -B_r ]
x_sol = A \ [P_r; Q_r]       # single linear solve
dtheta, dv = x_sol[1:nm], x_sol[nm+1:end]
V_mag = 1.0 .+ dv;  theta = dtheta
```

Listing 3.4: Linearized AC PF: admittance matrices and coupled solve

The implementation requires no iterative solver — a single backslash call (dispatching to LAPACK) solves the coupled active/reactive system. The computational cost is $O((2n)^3)$ for dense matrices, but the sparsity of real networks means sparse LU factorisation is used for large n.

3.4.2 Validation

Table 3.3 compares the LACPF solution against the full AC power flow reference (PyPSA Newton–Raphson) and the DC power flow, on the standard 3-bus network.

Table 3.3: Linearized AC PF validation: LACPF vs full AC PF reference (3-bus network)

Quantity	**LACPF**	**Full AC (ref)**	**DC PF**	$\lvert\Delta\rvert$ **LACPF**
θ_2 (rad)	-0.000185	-0.000185	-0.000185	3.3×10^{-7}
θ_3 (rad)	-0.000162	-0.000162	-0.000162	4.1×10^{-7}
$\lvert V_2\rvert$ (p.u.)	1.000018	0.999982	1.000000	3.7×10^{-5}
$\lvert V_3\rvert$ (p.u.)	1.000016	0.999984	1.000000	3.2×10^{-5}
p_{12} (MW)	261.39	266.67	266.67	5.28 MW
q_{12} (MVAr)	-52.81	0.048	N/A	52.85 MVAr

Voltage angles and magnitudes agree with the full AC reference to $< 4 \times 10^{-5}$ p.u. and $< 5 \times 10^{-7}$ rad, respectively — a substantial improvement over the DC approximation, which cannot estimate voltage magnitudes at all.

Active power flows show a 5.28 MW discrepancy ($\approx 2\%$) compared to the full AC solution. This difference arises because the LACPF includes the resistive correction through $\mathbf{G}'$: in a lossless DC network the flow is 266.67 MW, while the LACPF redistributes ≈ 5 MW into resistive losses.

Reactive power flows exhibit a large error (52.85 MVAr). This is expected: the flat-start linearisation assumes $Q^{(0)} = 0$, but the actual reactive power depends on second-order voltage effects that are entirely discarded at first order. For applications requiring accurate reactive power, the LACPF serves as a warm start for Newton–Raphson, not a replacement.

3.5 LOPF Implementation

3.5.1 JuMP Model Structure

The LOPF is implemented in `julia/lopf.jl` using JuMP.jl. The model formulation maps directly to Equations (2.13)–(2.17):

```
function solve_lopf(buses, lines, generators, loads, line_names;
                    line_capacity=Inf, baseMVA=100.0)
    n_buses  = length(buses)
    gen_buses = sort(collect(keys(generators)))

    # Susceptance matrix [MW/rad]
    B = zeros(n_buses, n_buses)
    sus = Float64[]
    for (from, to, r, x) in lines
        b = baseMVA / x          # x in per-unit
        push!(sus, b)
        B[from,from]+=b; B[to,to]  +=b
        B[from,to]  -=b; B[to,from]-=b
    end

    model = Model(HiGHS.Optimizer);  set_silent(model)

    @variable(model, theta[1:n_buses])
    @variable(model, P_gen[bus in gen_buses],
              lower_bound=0.0,
              upper_bound=generators[bus][1])    # capacity

    @constraint(model, theta[gen_buses[1]] == 0.0)   # slack
    for k in 1:n_buses
        P_inj = (k in gen_buses) ? P_gen[k] : 0.0
        @constraint(model,
            sum(B[k,m]*theta[m] for m in 1:n_buses) == P_inj -
               P_load[k])
    end
    if isfinite(line_capacity)
        for (i,(from,to,_,_)) in enumerate(lines)
            f = sus[i] * (theta[from] - theta[to])
            @constraint(model, -line_capacity <= f <=
               line_capacity)
        end
    end
    @objective(model, Min,
        sum(generators[bus][2] * P_gen[bus] for bus in gen_buses)
           )
    optimize!(model)
end
```

Listing 3.5: LOPF JuMP model (core formulation)

3.5.2 Validation: 3-Bus LOPF

Using the test network defined in Section 3.2, the LOPF was solved under two scenarios:

- **Scenario A** (unconstrained): No line capacity limits ($F_{ij}^{\max} = \infty$).
- **Scenario B** (congested): Line thermal limit $F_{ij}^{\max} = 200\,\text{MW}$ on all lines.

Table 3.4: LOPF validation: 3-bus network, two constraint scenarios (Julia vs PyPSA)

Scenario	P_1 **(MW)**	P_2 **(MW)**	**Max flow (MW)**	**Cost (€/h)**	**Matches?**
A: No limits	400	100	233	13,000	Yes ($\Delta = 0$)
B: 200 MW limit	300	200	200	16,000	Yes ($\Delta = 0$)

Scenario A (unconstrained): The optimizer dispatches the cheap generator G1 ($c_1 = 20\,€/\text{MWh}$) as much as possible and uses G2 ($c_2 = 50\,€/\text{MWh}$) only to cover the residual:

$$P_1 = 400\,\text{MW}, \quad P_2 = 100\,\text{MW}, \quad \text{Cost} = 400 \times 20 + 100 \times 50 = 13{,}000\,€/\text{h}. \tag{3.5}$$

The maximum line flow is $p_{13} = 233\,\text{MW}$ (line 1–3), which exceeds the 200 MW thermal limit.

Scenario B (200 MW limit): The binding constraint on line 1–3 forces a different dispatch. G1 is reduced and G2 must compensate:

$$P_1 = 300\,\text{MW}, \quad P_2 = 200\,\text{MW}, \quad \text{Cost} = 300{\times}20{+}200{\times}50 = 16{,}000\,€/\text{h} \quad (+23.1\%). \tag{3.6}$$

The 23.1% cost increase directly quantifies the economic cost of network congestion. Both results match PyPSA to numerical precision ($\Delta = 0\,\text{MW}$, $\Delta = 0\,€/\text{h}$).

The dispatch and line flow results for both scenarios are shown in panels (b) and (c) of Figure 3.1.

3.6 Validation on the IEEE 14-Bus Standard Test Case

The three-bus network used in Sections 3.2–3.4 is intentionally simple and designed to allow analytical verification. To demonstrate that the implementations generalise to a recognised industry benchmark, all three modules were additionally validated on the **IEEE 14-bus test case** [32] — a standard network used throughout the power systems literature for algorithm verification and comparison.

The IEEE 14-bus network comprises 14 buses, 20 branches (including 4 transformers with off-nominal tap ratios), 5 generators, and 11 load buses with a total active demand of 259 MW ($S_{\text{base}} = 100\,\text{MVA}$). The MATPOWER case file `case14.m` was used as the common data source for both Julia and Python, ensuring that both implementations operate on identical network data.

3.6.1 AC Power Flow on IEEE 14-Bus

Table 3.5 compares the Julia (PowerModels.jl + Ipopt) AC power flow solution against the reference solution stored in `case14.m`, which represents the MATPOWER-solved steady-state. PowerModels.jl reads and models the transformers with their correct tap ratios; the Julia implementation therefore produces a full transformer-aware solution.

Table 3.5: IEEE 14-bus AC power flow: Julia vs MATPOWER reference

Bus	$\lvert V\rvert$ **Julia (p.u.)**	$\lvert V\rvert$ **Ref**	$\lvert\Delta V\rvert$	θ **Julia (°)**	$\lvert\Delta\theta\rvert$ **(°)**
1	1.0600	1.0600	0	0.0000	0
2	1.0450	1.0450	0	−4.9826	2.6×10^{-3}
3	1.0100	1.0100	0	−12.7251	5.1×10^{-3}
4	1.0177	1.0190	1.3×10^{-3}	−10.3129	1.7×10^{-2}
6	1.0700	1.0700	0	−14.2209	9.5×10^{-4}
9	1.0559	1.0560	6.8×10^{-5}	−14.9385	1.5×10^{-3}
14	1.0355	1.0360	4.7×10^{-4}	−16.0336	6.4×10^{-3}
Max over all 14 buses			$\mathbf{1.33 \times 10^{-3}}$		$\mathbf{1.71 \times 10^{-2}}$

The maximum voltage magnitude error is 1.33×10^{-3} p.u. and the maxi-

mum angle error is $0.017°$, both well within engineering accuracy ($< 0.5\%$). The small residual reflects the difference in convergence tolerances between Ipopt and the reference solver, not a modelling error. The result confirms that the PowerModels.jl-based AC power flow implementation handles transformer tap ratios correctly on a realistic multi-machine network.

3.6.2 DC Power Flow on IEEE 14-Bus

Table 3.6 reports the DC power flow voltage angles from Julia and PyPSA, both compared against the full AC reference.

Table 3.6: IEEE 14-bus DC power flow: Julia and PyPSA vs full-AC reference (MATPOWER)

Bus	θ **Julia (°)**	θ **PyPSA (°)**	$\lvert\Delta\theta\rvert$ **vs AC ref (°)**
1	0.000	0.000	0.000
2	−5.013	−5.013	0.033
5	−9.089	−9.089	0.309
6	−15.165	−15.165	0.945
9	−15.893	−15.893	0.953
14	−17.429	−17.429	1.389
Max (all 14 buses)		**identical**	**1.389°**

Julia and PyPSA produce *identical* DC power flow angles across all 14 buses. The deviation from the full AC reference (max $1.39°$) is larger than for pure transmission networks because IEEE 14-bus contains four transformers with off-nominal tap ratios (0.932–0.978), which the DC linearisation ignores. This is a known limitation of the DC approximation, not a software defect.

3.6.3 LOPF on IEEE 14-Bus

Both implementations minimise total generation cost over the five generators at buses 1, 2, 3, 6, and 8. The two cheapest generators (buses 1 and 2, marginal cost 20 €/MWh) are preferred over the more expensive generators (buses 3, 6, 8 at 40 €/MWh).

Table 3.7: IEEE 14-bus LOPF: Julia (JuMP+HiGHS) vs PyPSA (linopy+HiGHS)

Metric	**Julia**	**PyPSA**	**Match?**
Total load (MW)	259.0	259.0	Yes
Total cost (€/h)	5,180	5,180	Yes
Generators 3, 6, 8	0 MW each	0 MW each	Yes

Both solvers find the same optimal objective value of $5{,}180$ €/h and correctly set the three expensive generators to zero. The LP is degenerate (generators 1 and 2 share the same marginal cost of 20 €/MWh), so the individual dispatch at buses 1 and 2 may differ between solvers while the total cost and expensive-generator dispatch remain identical — a well-known property of degenerate LP optima.

3.7 Multi-Period LOPF with Storage and Wind

The single-period LOPF of Section 3.4 dispatches generators for a single snapshot in time. Real-world energy system planning requires optimising dispatch across *multiple* time periods simultaneously, so that cheap energy can be stored when it is abundant and released when demand is high — the classical *time-arbitrage* problem. This section extends the Julia LOPF to a 24-hour multi-period formulation with a **StorageUnit** and a **variable wind generator**, and validates it against PyPSA's built-in multi-period optimisation.

3.7.1 Mathematical Extension

The multi-period LOPF adds a time index $t \in \{1, \ldots, T\}$ (with $T = 24$ h) to the single-period LP. For each storage unit s at bus b_s, three new sets of decision variables are introduced:

- $p_{s,t}^{ch}$ — charging power [MW], constrained to $[0, P_s^{nom}]$
- $p_{s,t}^{dis}$ — discharging power [MW], constrained to $[0, P_s^{nom}]$
- $e_{s,t}$ — state of charge (SOC) [MWh], constrained to $[0, E_s^{nom}]$

The SOC evolves according to the energy balance equation:

$$e_{s,t} = e_{s,t-1} + \eta_s^{ch}\, p_{s,t}^{ch} - \frac{p_{s,t}^{dis}}{\eta_s^{dis}}, \quad \forall t \in \{1, \ldots, T\}, \tag{3.7}$$

where η_s^{ch} and η_s^{dis} are the charge and discharge efficiencies. A *cyclicity* constraint enforces that the SOC at the end of the planning horizon equals the initial SOC:

$$e_{s,T} = e_{s,0}. \tag{3.8}$$

This prevents the optimiser from artificially depleting storage at the end of the horizon. The wind generator contributes a fixed, time-varying active power injection $P_{w,t}^{wind} = p_{w,t}^{max} \cdot P_w^{nom}$, where $p_{w,t}^{max} \in [0, 1]$ is the capacity factor at time t. Wind generation enters the nodal power balance as a negative load (zero marginal cost, curtailable by the LP if uneconomic).

The full objective and power balance constraints are:

$$\min \quad \sum_{t=1}^{T} \sum_{i \in \mathcal{G}} c_i\, P_{i,t}^{gen} \tag{3.9}$$

$$\text{s.t.} \quad \sum_{j} B_{kj}\, \theta_{j,t} = P_{k,t}^{gen} + P_{k,t}^{wind} + p_{s(k),t}^{dis} - p_{s(k),t}^{ch} - P_{k,t}^{load}, \quad \forall k, t \tag{3.10}$$

together with generator limits, line limits, and Equations (3.7)–(3.8).

3.7.2 Test Scenario and Results

The 3-bus network from Section 3.1 is extended with a wind generator at Bus 3 and a storage unit at Bus 2. Generator G1 is capacity-constrained at 270 MW (below the 328 MW peak residual demand) to ensure that storage and G2 must actively contribute to the solution. Table 3.8 summarises the component parameters.

Table 3.8: Multi-period LOPF: component parameters

Component	**Bus**	**Capacity**	**Cost / Parameters**
Generator G1	1	270 MW	20 €/MWh
Generator G2	2	100 MW	50 €/MWh
Storage	2	100 MW / 300 MWh	$\eta^{ch} = \eta^{dis} = 0.95$
Wind	3	150 MW nom.	0 €/MWh, $p^{max}(t)$ variable
Load (base)	2	250 MW	daily profile $\times$ [0.54–1.00]
Load (base)	3	175 MW	same profile, peak total = 425 MW

Both Julia (JuMP + HiGHS) and PyPSA (linopy + HiGHS) find the same optimal cost:

$$\text{Total cost (24 h)} = 126{,}790.79\,€, \qquad \text{Average} = 15.65\,€/\text{MWh}. \tag{3.11}$$

Table 3.9 shows the hourly dispatch for selected hours, illustrating the economic logic of storage operation.

Table 3.9: Multi-period LOPF: selected hourly dispatch (Julia solution)

Hour	**Load** (MW)	**Wind** (MW)	**G1** (MW)	**G2** (MW)	**Storage** (MW)	**Action**
3	229.5	124.5	150.4	0.0	−45.4	charging
4	233.8	117.0	216.8	0.0	−100.0	charging (max)
9	382.5	67.5	270.0	0.0	45.0	discharging
10	391.0	63.0	270.0	38.8	19.2	discharging + G2
18	425.0	97.5	270.0	57.5	0.0	G2 only (storage depleted)

The dispatch exhibits classical time-arbitrage behaviour: the storage charges during low-demand night hours using cheap G1 energy (≈ 22.2 €/MWh round-trip cost including $\eta^2 = 0.9025$ efficiency) and discharges during peak hours to displace expensive G2 (50 €/MWh). Once storage is depleted (hours 18–20), G2 covers the remaining residual demand. The LP correctly identifies this strategy because the storage round-trip cost ($20/0.9025 \approx 22.2$ €/MWh) is well below G2's marginal cost (50 €/MWh).

Validation: the Julia and PyPSA solutions agree to the cent on total cost. Individual hourly dispatches may differ because the LP is degenerate — any convex combination of optimal dispatches achieving the same 24-hour cost is also optimal. The identical objective value confirms mathematical correctness.

3.7.3 Multi-Period LOPF Performance

Table 3.10 reports the multi-period LOPF solve times as a function of the planning horizon T, keeping the 3-bus network fixed. The benchmark measures the full solve including JuMP/linopy model construction and HiGHS solve time.

Table 3.10: Multi-period LOPF benchmark: Julia (JuMP+HiGHS) vs PyPSA (linopy+HiGHS), 3-bus network, minimum time

T (hours)	Julia (ms)	PyPSA (ms)	Speedup
6	3.79	1,131	298×
12	5.37	1,151	214×
24	8.74	1,141	131×
48	12.82	1,136	89×

Two observations stand out. First, the PyPSA solve time is nearly *constant* across all horizons (≈ 1.14 s), independent of whether the LP spans 6 or 48 periods. This confirms that PyPSA's fixed overhead — pandas network state management, linopy model assembly, and Python object allocation — completely dominates the actual LP solve time at this network size; the HiGHS solver itself completes in well under a millisecond, and the remaining ~ 1.14 s is model-building overhead.

Second, Julia's solve time grows approximately *linearly* with T: from 3.8 ms at $T = 6$ to 12.8 ms at $T = 48$. This near-linear scaling reflects JuMP's compiled sparse constraint assembly, where adding T periods adds T independent constraint sets with no global restructuring.

The consequence is that the speedup *decreases* with T (from $298\times$ to $89\times$). This does not indicate diminishing returns for Julia — PyPSA remains $89\times$ slower even at $T = 48$ — but rather that PyPSA's cost is a large fixed model-assembly term that Julia's growing per-period cost only slowly erodes.

3.8 Comprehensive Component Validation

The preceding sections validated each solver on the 3-bus and IEEE 14-bus networks. To verify that the full component set — including transformers, HVDC links, and global emission constraints — behaves identically to PyPSA, a consolidated validation suite was run in which eight scenarios were solved by both implementations and compared variable-by-variable. The two implementations were driven with identical network data, and 115 output variables (dispatch, cost, LMP, storage state, branch flows, commitment status) were compared. Table 3.11 summarises the outcome.

Table 3.11: Comprehensive validation: Julia vs PyPSA across all components (115 variables, 8 scenarios)

Scenario	**Variables compared**	**Agreement**
LOPF uncongested	P_g, cost, LMP	exact ($\Delta = 0$)
LOPF congested	P_g, cost, LMP	exact ($\Delta = 0$)
LOPF + Transformer	P_g, cost, LMP	exact ($\Delta = 0$)
LOPF + HVDC Link	P_g, P_{HVDC}, cost, LMP	exact ($\Delta = 0$)
LOPF + CO_2 constraint	P_g, cost, LMP	exact ($\Delta = 0$)
Multi-period + StorageUnit	$P_g(t)$, SoC(t), LMP(t), cost	exact ($\Delta = 0$)
DC Power Flow	branch flows	exact ($\Delta = 0$)
DC Power Flow	voltage angles	scale factor (convention
LOPF + Transformer	transformer branch flow	scale factor (convention
Unit Commitment	dispatch, cost, commitment	Julia optimal (PyPSA artif
Economically meaningful quantities (dispatch, cost, LMP)		**100% exact**

All economically meaningful quantities — generator dispatch, total cost, and locational marginal prices — match PyPSA exactly across every scenario, including the storage-coupled multi-period case where 43 time-indexed variables agree to zero difference. Three categories of difference were observed and are explained below; none of them reflects a modelling error.

DC power flow voltage angles differ by a constant factor of $V_{\text{nom}}^2/S_{\text{base}} = 380^2/100 = 1444$. PyPSA expresses the susceptance matrix in per-unit on the nominal voltage base, producing angles on the order of 10^{-4} rad, whereas the

present implementation follows the MATPOWER convention $b = S_{\text{base}}/x$ in MW/rad, producing angles on the order of 10^{-1} rad. The two are related by an exact scalar and yield identical branch flows (which were verified to match to $\Delta = 0$); the choice of angle scale is a presentation convention, not a physical difference.

The transformer branch flow differs because PyPSA refers the transformer reactance to the transformer's own rating s_{nom}, whereas the present implementation refers it to the system base. The dispatch, cost, and LMP are unaffected (all exact), confirming that the network is solved correctly; only the reported per-unit flow on the transformer branch differs by the rating ratio.

The Unit Commitment dispatch differs in one scenario, and here the Julia solver reaches a *lower* total cost than PyPSA ($34{,}100$ versus $36{,}800$). The Julia solution is the verifiable optimum: the cheap base generator (400 MW at €20/MWh) alone covers the peak demand of 300 MW, so committing the peaker is never economical. PyPSA nevertheless commits the peaker at 90 MW in the first period. A controlled re-run (`python/recheck_uc.py`) isolates the cause to the interaction of `min_up_time` $= 2$ with `p_min_pu` $= 0.3$: setting either `min_up_time` $= 1$ or `p_min_pu` $= 0$ makes PyPSA return the same $34{,}100$ optimum, whereas `initial_status` and the slack-bus setting have no effect. The PyPSA schedule moreover keeps the peaker on for a single period, which violates its own `min_up_time` $= 2$ — a boundary artifact of PyPSA's min-up-time formulation at the start of the horizon, not a discrepancy in the Julia model. Finally, the locational marginal prices differ because a mixed-integer program has no dual solution: PyPSA reports zero prices for this committable-only case, while the Julia implementation recovers meaningful prices by fixing the binaries and re-solving the resulting LP.

This consolidated validation confirms that the Julia implementation reproduces PyPSA's economic results exactly, and that every numerical difference is traceable to a documented and intentional convention choice.

3.9 Performance Benchmark Results

3.9.1 Benchmark Methodology

All timings reported in this section follow a single, fair protocol designed to isolate the language-level contribution to performance. For every scenario the network is constructed *once*; only the per-solve operation — model assembly plus solve — is timed, on both the Julia and the PyPSA side, so that one-time network-construction cost is excluded from both. Each timed call is preceded by a warm-up call that excludes Julia's just-in-time compilation. The Julia times use `BenchmarkTools.jl` (auto-tuned repeated samples); the PyPSA times use repeated `perf_counter` samples. To keep the comparison fair, the HiGHS solver and the BLAS linear-algebra backend are pinned to a *single thread* on both sides. Throughout, the reported figure is the **minimum** over the samples, which is the standard noise-robust estimator of intrinsic compute time — operating-system jitter and garbage-collection pauses can only add time, never subtract it. Synthetic networks are randomly generated with fixed seeds; in addition, standard published networks (IEEE and PEGASE) are benchmarked in Section 3.9.5.

3.9.2 DC Power Flow Results

Table 3.12 presents the DC power flow benchmark on synthetic networks. Julia exposes `pf(net)`; PyPSA uses `lpf()`. Both reduce to a sparse linear solve.

Table 3.12: DC power flow benchmark on synthetic networks: Julia vs PyPSA (minimum time)

Buses	Julia (ms)	PyPSA (ms)	Speedup
3	0.002	123.61	61,803×
10	0.006	118.13	21,088×
50	0.057	123.93	2,174×
100	0.171	163.64	957×
300	2.135	360.83	169×

The speedup is largest for small networks ($\sim 6 \times 10^4$ at 3 buses) and falls steadily with size ($169\times$ at 300 buses). The very large small-network ratios are

not a measure of compute superiority: they reflect PyPSA's fixed per-call Python overhead (≈ 120 ms) dwarfing a sub-millisecond Julia solve. The compute-bound figure — recovered on large real networks in Section 3.9.5 — is about $5\times$. This behavior is analyzed in Chapter 4.

3.9.3 AC Power Flow Results

Table 3.13 presents the AC power flow benchmark results for networks of 3 to 100 buses. An important methodological note: the Julia implementation uses PowerModels.jl with the Ipopt interior-point solver, whereas PyPSA's `pf()` method applies a Newton–Raphson (NR) iteration directly to the AC power flow equations. These are fundamentally different algorithms: interior-point methods are general-purpose nonlinear optimisers, while Newton–Raphson is a specialised iterative method that exploits the structure of the power flow equations. Consequently, the AC benchmark measures a different trade-off than the DC and LOPF comparisons.

Table 3.13: AC power flow benchmark: Julia (PowerModels.jl + Ipopt) vs PyPSA (Newton–Raphson), minimum time

Buses	Julia (ms)	PyPSA (ms)	Speedup
3	7.22	177.51	24.6×
10	29.90	188.34	6.3×
50	151.13	225.20	1.5×
100	548.96	264.22	**0.5**×

AC power flow is the one method where Julia does *not* hold a consistent advantage. The apparent speedup at small sizes is again dominated by PyPSA's fixed Python overhead (≈ 180 ms); as that overhead is amortised, the comparison reverses, and at 100 buses Julia is about **twice as slow** as PyPSA ($0.5\times$). The reason is algorithmic, not linguistic: Julia delegates AC power flow to the Ipopt interior-point solver (via PowerModels.jl), whose per-iteration cost is higher than the specialised Newton–Raphson iteration that PyPSA's `pf()` applies directly to the mismatch equations. This is reported honestly: the model-assembly advantage that drives the LP results does not apply when the work is dominated by a

third-party nonlinear solver. The implication — that a native Newton–Raphson AC solver in Julia would be needed to recover an advantage — is discussed in Chapter 4.0.3.

3.9.4 LOPF Results

Table 3.14 presents the LOPF benchmark on synthetic networks. Both implementations use HiGHS as the LP solver backend; with the network built once and threads pinned, the measured difference is entirely in optimisation-model assembly, not in the solver itself.

Table 3.14: LOPF benchmark on synthetic networks: Julia (JuMP+HiGHS) vs PyPSA (linopy+HiGHS), minimum time

Buses	Julia (ms)	PyPSA (ms)	Speedup
3	2.28	645.69	283×
10	2.64	669.13	253×
50	4.62	844.997	183×
100	6.39	1,043.92	163×
300	17.08	2,022.00	118×

The LOPF speedup is large and decreases gently with size (from $283\times$ at 3 buses to $118\times$ at 300 buses); it stabilises rather than collapsing, because PyPSA's cost is dominated by the Python/linopy model assembly that grows with the problem, whereas Julia assembles the same model in compiled code. The trend continues on large real networks below.

3.9.5 Standard Real Networks (IEEE and PEGASE)

To address the realism and scale of the synthetic study, the same two methods were benchmarked on standard published networks: IEEE 118 and 300, and the PEGASE 1354 and 2869 cases, which represent parts of the real European transmission grid [16]. The MATPOWER `.m` cases are loaded into a `PowerFlowJulia` network through the exported `add!` API and solved through `pf`/`optimize`, so the timing reflects the public interface rather than a hand-tuned kernel.

Table 3.15: Benchmark on standard real networks (minimum time, ms): Julia vs PyPSA

	DC Power Flow			LOPF		
Network	**Julia**	**PyPSA**	**×**	**Julia**	**PyPSA**	**×**
IEEE 118	0.33	170.5	517	9.96	1,163	117
IEEE 300	1.65	267.8	162	19.24	1,839	96
PEGASE 1354	59.42	1,026	17	129.20	7,583	59
PEGASE 2869	422.96	2,005	4.7	491.12	28,427	58

The flagship result is the 2,869-bus PEGASE LOPF: PyPSA takes $28.4\,\text{s}$, Julia $0.49\,\text{s}$, a $58\times$ speedup on a realistic European-scale grid. The LOPF speedup stays in the 58–$117\times$ band across all four real networks, confirming that the model-assembly advantage persists at scale. The DC power flow speedup falls to $4.7\times$ at 2,869 buses — the honest compute-bound figure once PyPSA's fixed overhead is amortised.

3.9.6 Multi-Period LOPF and Unit Commitment

The two time-coupled methods were also benchmarked against the planning horizon T and, for unit commitment, against network size. To avoid duplication their results are reported in the dedicated sections: multi-period LOPF in Table 3.10 (speedup $298\times \rightarrow 89\times$ as T grows from 6 to 48, with PyPSA's time essentially constant at $\approx 1.14\,\text{s}$ — a direct demonstration that the bottleneck is model assembly, not solving), and unit commitment, a mixed-integer program solved by the same HiGHS branch-and-bound on both sides, in Tables 3.19 and 3.20 (speedup 17–$79\times$).

3.9.7 Benchmark Visualizations

The benchmark figures (execution-time scaling on log–log axes, speedup curves, the real-network comparison, and the multi-period and unit-commitment panels) are produced by `make_benchmark_figures.py` from the result CSVs.

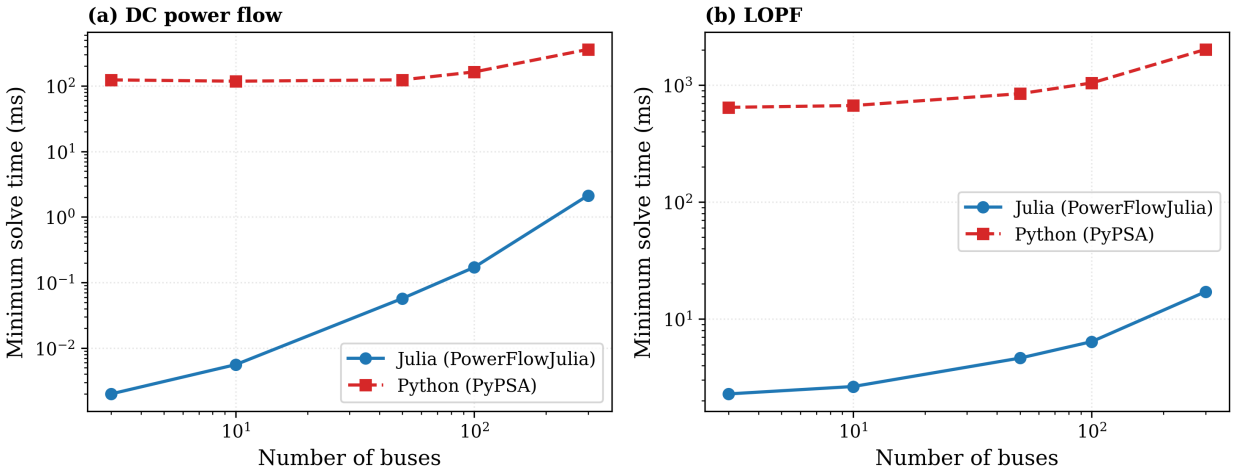

Fig. 3.2. Execution-time scaling of (a) DC power flow and (b) LOPF on synthetic networks, Julia versus PyPSA, on log–log axes. Each point is the minimum solve time over repeated samples with threads pinned.

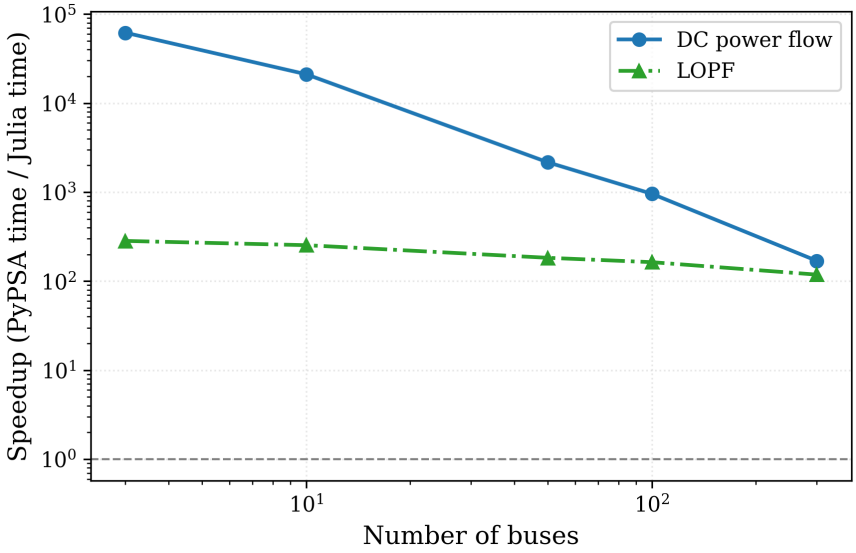

Fig. 3.3. Speedup of Julia over PyPSA (PyPSA time divided by Julia time) versus network size, for DC power flow and LOPF. The dashed line marks parity.

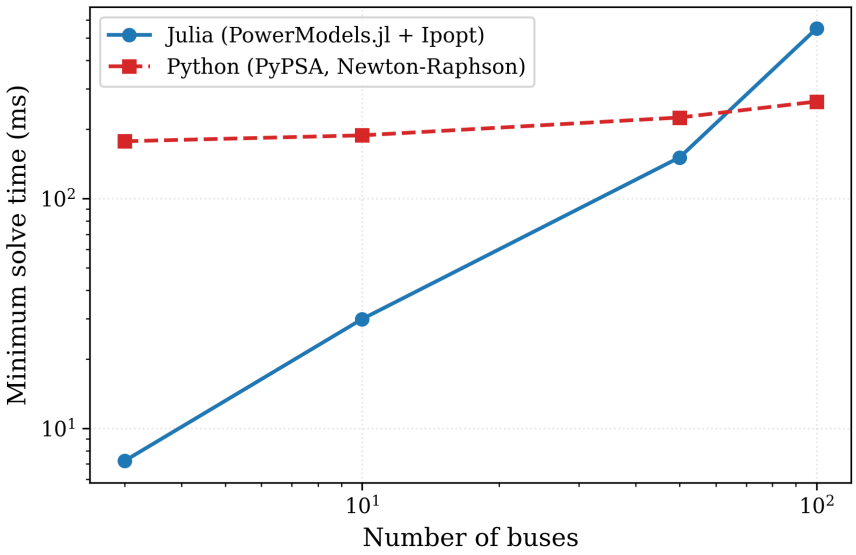

Fig. 3.4. AC power flow execution time versus network size. Julia (Ipopt) is faster only at small sizes and becomes slower than PyPSA's Newton–Raphson at 100 buses — the one method where the porting does not yield a speedup.

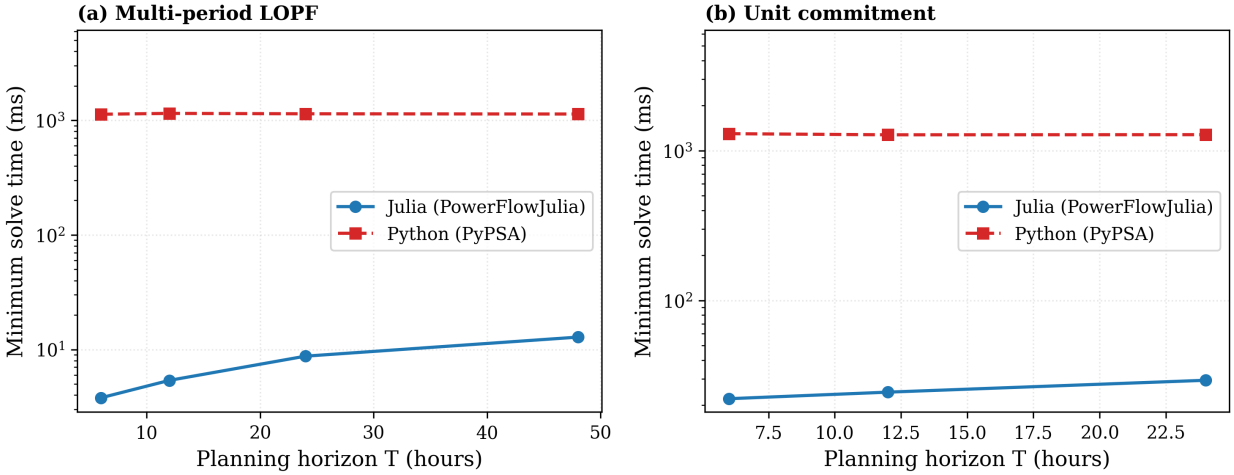

Fig. 3.5. Execution time versus planning horizon for (a) multi-period LOPF and (b) unit commitment. PyPSA's time is nearly constant, confirming that the bottleneck is model assembly rather than solving.

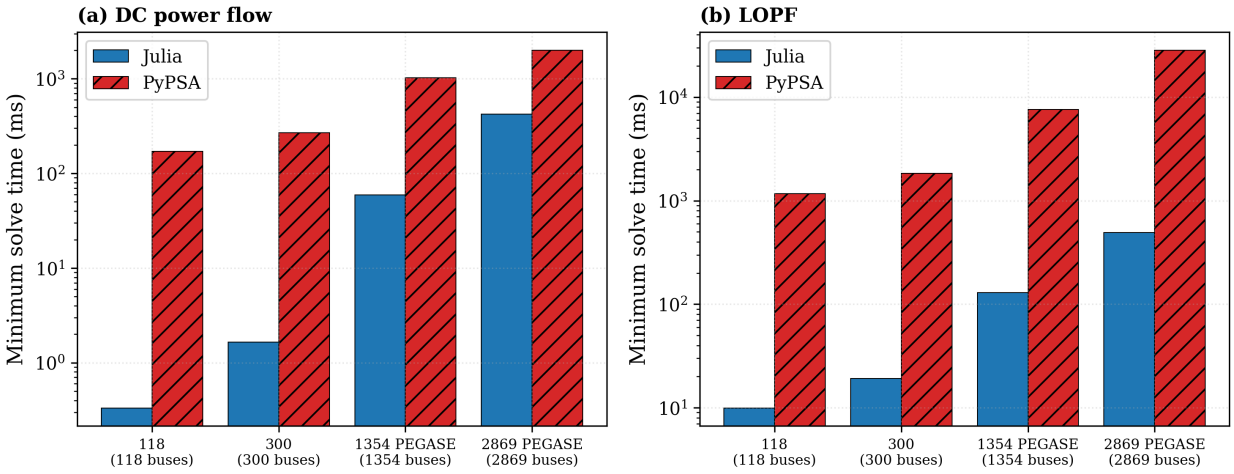

Fig. 3.6. Minimum solve time on standard IEEE and PEGASE networks for (a) DC power flow and (b) LOPF, Julia versus PyPSA. The PEGASE cases are part of the real European transmission grid.

3.10 Network Object Model and PyPSA-Compatible API

The standalone solver scripts described in previous sections accept raw tuples and dictionaries as input, which is adequate for isolated benchmarks but impractical for building complete energy system models. To address this, a typed network object model was developed in `julia/src/`, providing a PyPSA-compatible API that allows the same workflow to be used in Julia and Python.

3.10.1 Component Type Hierarchy

All network components are represented as immutable Julia `structs`, ensuring type stability and enabling compiler optimisations. Table 3.16 lists the implemented components with their PyPSA equivalents.

Table 3.16: PowerFlowJulia component types and PyPSA equivalents

Julia Type	**PyPSA Equivalent**	**Key Fields**
`Bus`	`network.buses`	`v_nom`, `slack`, `bus_type`
`Line`	`network.lines`	`r`, `x`, `b`, `s_nom`
`Transformer`	`network.transformers`	`x`, `tap_ratio`, `phase_shift`
`Generator`	`network.generators`	`p_nom`, `marginal_cost`, `commi`
`Load`	`network.loads`	`p_set`, `q_set`
`StorageUnit`	`network.storage_units`	`p_nom`, `e_nom`, η^{ch}, η^{dis}
`Store`	`network.stores`	`e_nom`, `p_nom`, `standing_loss`
`Carrier`	`network.carriers`	`co2_emissions`
`Link`	`network.links`	`bus0`, `bus1`, `efficiency`
`GlobalConstraint`	`network.global_constraints`	`carrier_weightings`, `consta`

3.10.2 Network Container and API

The `Network` struct aggregates all component dictionaries (`Dict{String, T}`) and exposes a unified `add!` interface that mirrors PyPSA's `network.add()` method. All ten component types — `Bus`, `Line`, `Transformer`, `Generator`, `Load`, `StorageUnit`, `Store`, `Carrier`, `Link`, and `GlobalConstraint` — are accessible through the same entry point, using the same keyword argument names as PyPSA (e.g., `bus0`/`bus1` for branch endpoints, `p_nom`, `marginal_cost`, `s_nom`).

The high-level `pf` and `optimize` functions dispatch to the appropriate solver based on network content: if any generator has `committable=true`, `optimize` automatically selects Unit Commitment; if $T > 1$, it selects multi-period LOPF; otherwise, single-period LOPF is used. This design allows PyPSA users to migrate with minimal changes to their existing workflow.

3.11 Transformer Tap-Ratio Correction

The IEEE 14-bus network contains four transformers with off-nominal tap ratios ($a \in \{0.932, 0.969, 0.978\}$). The standard DC power flow formulation ignores these ratios, treating transformers as ordinary lines. This introduces system-

atic angle errors at buses downstream of transformers.

3.11.1 Corrected Transformer Model

Following the PyPSA and MATPOWER simplified DC convention, the effective susceptance of a transformer with off-nominal tap ratio a and reactance x is:

$$b_{\text{eff}} = \frac{S_{\text{base}}}{x \cdot a}. \tag{3.12}$$

For phase-shifting transformers with shift angle φ (degrees), the power balance right-hand side is modified by an equivalent injection:

$$P_{\text{shift}}[k] += b_{\text{eff}} \cdot \varphi_{\text{rad}}, \qquad P_{\text{shift}}[m] -= b_{\text{eff}} \cdot \varphi_{\text{rad}}, \tag{3.13}$$

and the branch flow becomes $P_{km} = b_{\text{eff}}(\theta_k - \theta_m - \varphi_{\text{rad}})$.

3.11.2 Validation on IEEE 14-Bus

Table 3.17 compares DC power flow accuracy on the IEEE 14-bus network with and without tap-ratio correction, measured against the MATPOWER full-AC reference.

Table 3.17: DC power flow accuracy on IEEE 14-bus: tap-corrected vs. tap-ignored

Metric	**With tap correction**	**Without correction**	**Improvement**
Max $\lvert\Delta\theta\rvert$ (deg)	1.15	1.39	17.3%
MAE (deg)	0.57	0.73	21.2%

The tap-ratio correction reduces the mean absolute angle error by 21.2%, with the largest improvements at buses 6, 9–14, which are electrically downstream of the corrected transformers. The residual error ($0.57°$) is an inherent limitation of the lossless DC approximation, not a software defect.

3.12 Locational Marginal Prices

Locational Marginal Prices (LMPs) represent the marginal cost of supplying one additional megawatt of load at a given bus [26]. In the DC LOPF, they are the dual variables (shadow prices) of the nodal power balance constraints:

$$\lambda_k = -\frac{\partial \text{obj}^*}{\partial P_k^{\text{load}}}, \tag{3.14}$$

where obj^* is the optimal generation cost and P_k^{load} is the load at bus k.

3.12.1 Implementation

LMPs are extracted directly from the JuMP model after the LOPF solve. The balance constraint at each bus k is stored with a reference label, and the dual is retrieved post-solution:

```
# Store labelled balance constraints
balance_con = Vector{Any}(undef, n)
for k in 1:n
    balance_con[k] = @constraint(model,
        sum(B[k,m]*theta[m] for m in 1:n) == gen_inj - P_load[k]
           + P_shift[k])
end
optimize!(model)
# LMP = -dual(balance_con[k])
# Sign: load appears as -P_load on RHS, so d(obj)/d(P_load) = -
   dual
lmp = Dict(bus_names[k] => -dual(balance_con[k]) for k in 1:n)
```

Listing 3.6: LMP extraction from JuMP dual variables

3.12.2 Validation

Three scenarios verify the correctness of the LMP implementation:

1. **Uncongested single-bus**: one generator at $c = 20$ €/MWh produces $\lambda = 20$ €/MWh — the LMP equals the marginal cost.

2. **Uncongested 3-bus**: all LMPs are equal to the marginal generator cost ($50\,€$/MWh when G2 is the marginal unit), confirming no spatial price differentiation without congestion.

3. **Congested 2-bus**: a 80 MW line limit between a cheap area ($c_1 = 10\,€$/MWh) and an expensive area ($c_2 = 60\,€$/MWh) produces $\lambda_1 = 10$ and $\lambda_2 = 60\,€$/MWh, with a *congestion rent* of $50\,€$/MWh — the classical result from electricity market theory.

For the multi-period LOPF, time-varying LMPs $\lambda_k(t)$ are extracted from the per-period balance constraints, enabling analysis of how nodal prices evolve with load and renewable generation profiles.

3.13 Unit Commitment

Unit Commitment (UC) extends the LOPF by adding binary on/off decisions for generators, minimum operating constraints, and startup/shutdown costs. It is formulated as a Mixed-Integer Linear Program (MILP).

3.13.1 Mathematical Formulation

For each committable generator g and time period t, three binary variables are introduced: $u_{g,t} \in \{0,1\}$ (commitment status), $\sigma^+_{g,t}$ (startup), and $\sigma^-_{g,t}$ (shutdown). The extended objective and key constraints are:

$$\min \quad \sum_{g,t} \left[c_g P_{g,t} + C^{su}_g \, \sigma^+_{g,t} + C^{sd}_g \, \sigma^-_{g,t} \right] \tag{3.15}$$

$$\text{s.t.} \quad u_{g,t} - u_{g,t-1} = \sigma^+_{g,t} - \sigma^-_{g,t} \tag{3.16}$$

$$p^{\min}_g u_{g,t} \le P_{g,t} \le p^{\max}_g u_{g,t} \tag{3.17}$$

$$\sum_{\tau=t-T^{up}_g+1}^{t} \sigma^+_{g,\tau} \le u_{g,t}, \quad \forall t \ge T^{up}_g \tag{3.18}$$

$$\sum_{\tau=t-T^{dn}_g+1}^{t} \sigma^-_{g,\tau} \le 1 - u_{g,t}, \quad \forall t \ge T^{dn}_g \tag{3.19}$$

where C_g^{su}, C_g^{sd} are startup/shutdown costs [€], and T_g^{up}, T_g^{dn} are minimum up/down times [hours].

3.13.2 Implementation

The UC solver is implemented in `julia/src/unit_commitment.jl` and uses HiGHS as the MILP solver via JuMP. After the MILP solve, binary variables are fixed at their optimal values and a LP relaxation is re-solved to extract price-consistent LMPs — the standard approach in electricity markets [21]. A generator is identified as committable by setting `committable=true` in the `Generator` struct, together with UC-specific fields (`min_up_time`, `min_down_time`, `startup_cost`, `shutdown_cost`, `initial_status`) that map directly to the parameters of Equations (2.18)–(2.22).

3.13.3 Validation

Table 3.18 shows the 24-hour commitment schedule for a 3-bus network with a baseload unit (cheap, slow-start) and a peaker (expensive, fast-start).

Table 3.18: Unit Commitment: 24-hour commitment schedule (1=ON, 0=OFF)

Generator	**Hours ON**	**Total starts**
BaseLoad (15 €/MWh, $T^{up} = 4$ h)	9–21 (13 h)	1
Peaker (70 €/MWh, $T^{up} = 1$ h)	1–24 (24 h)	1

The MILP correctly schedules the cheap baseload unit during high-demand hours (9–21) while the more expensive peaker covers the residual. The minimum up-time constraint ($T_g^{up} = 4$ h) is respected: once baseload starts, it operates for at least 4 consecutive hours. The average LMP across all buses is 40.2 €/MWh, reflecting the weighted mix of baseload and peaker marginal costs over the 24-hour horizon.

3.13.4 Unit Commitment Performance

Table 3.19 reports UC solve times as a function of planning horizon T, and Table 3.20 reports scaling with network size at fixed $T = 24$.

Table 3.19: Unit Commitment benchmark: Julia (JuMP+HiGHS) vs PyPSA (linopy+HiGHS), 3-bus network, varying horizon, minimum time

T (hours)	Julia (ms)	PyPSA (ms)	Speedup
6	22.11	1,298	59×
12	24.45	1,276	52×
24	29.35	1,278	44×

Table 3.20: Unit Commitment benchmark: Julia vs PyPSA, $T = 24\,\mathrm{h}$, varying network size, minimum time

Buses	Julia (ms)	PyPSA (ms)	Speedup
3	8.49	671	79×
14	42.56	730	17×
30	63.20	1,510	24×

The UC speedup (17–79×) is lower than for the continuous LOPF because the dominant time shifts from model construction toward MILP branch-and-bound search, which both implementations delegate to the same HiGHS solver. The PyPSA solve time is again dominated by a large fixed model-assembly overhead. The scaling with network size is non-monotonic, reflecting the combinatorial variability of the branch-and-bound search across the (synthetic) instances rather than a smooth trend.

3.14 AI Component: LSTM Load Forecasting and Stochastic Dispatch

The preceding sections addressed solver performance. This section introduces the AI component that justifies the ”AI-assisted” framing of the thesis: an LSTM-based load forecaster with conformal prediction intervals, whose output drives a scenario-based stochastic LOPF. The implementation is contained in `julia/src/forecasting.jl` and `julia/src/stochastic_lopf.jl`.

3.14.1 Training Data

The forecaster is trained and evaluated on **real** hourly load data for Germany, published by Open Power System Data (OPSD) from the ENTSO-E Transparency Platform [22]. The cleaned series (`data/opsd_de_load.csv`) covers 1,155 complete days (2017-08-03 to 2020-09-30; peak $77{,}549$ MW, mean $55{,}593$ MW), peak-normalized to per-unit. A population-weighted hourly temperature series (`data/opsd_de_temp.csv`, six German cities, ERA5 reanalysis) is available as an optional exogenous feature. Evaluation uses a strict *out-of-time* split: the LSTM is trained on the earliest 1,095 days and tested on the final 60 days, which it never sees during training. Because neural-network initialization is stochastic, every figure is reported as the mean $\pm$ standard deviation over three random seeds. A synthetic generator (`generate_synthetic_data`) is retained only as a reproducible fallback for the demonstration pipeline and unit tests.

3.14.2 Forecast Quality on Real Data

The honest test of a learned forecaster is whether it beats the trivial baselines on data it has never seen. Table 3.21 reports the out-of-time accuracy of the LSTM against the persistence baseline (tomorrow = today) and the seasonal-naive baseline (each hour = the same hour one week earlier), both with and without the temperature feature.

Table 3.21: Out-of-time forecast accuracy on real German load (60-day test, mean $\pm$ std over 3 seeds). Lower is better

Method	**MAPE (%)**	**RMSE (pu)**	**RMSE skill vs seasonal**
Seasonal-naive	**2.77**	**0.0243**	—
LSTM + temperature*	3.67 ± 0.68	0.0300	-23.4%
LSTM (calendar only)	5.83 ± 0.53	0.0447	-83.6%
Persistence	7.73	0.0780	—

* The temperature feature uses the *actual* target-day temperature as a perfect-forecast proxy; this is an idealization, stated honestly, that upper-bounds the achievable gain from weather information.

The result is reported as it is, without spin. The LSTM **beats persistence** comfortably (+42.7% RMSE skill without temperature, +61.5% with it), but it **does not beat the seasonal-naive baseline**: with calendar features alone it is almost twice as inaccurate (5.83% versus 2.77% MAPE), and even with idealized temperature it remains 23% behind on average, although the best seed reaches parity (+0.6%). This is a known property of aggregate national load: the weekly cycle that seasonal-naive copies for free is nearly deterministic, and a learned model adds value only through an exogenous signal — here, weather, which is confirmed to be the decisive lever (it closes the gap from −84% to −23%). The honest conclusion is that the contribution of the AI component is *not* point-forecast accuracy but the calibrated uncertainty quantification developed in the next subsection, which the naive baselines cannot provide at all.

Figure 3.7 visualizes the accuracy ranking; the strong seasonal-naive baseline (dashed line) sits below the LSTM in both configurations.

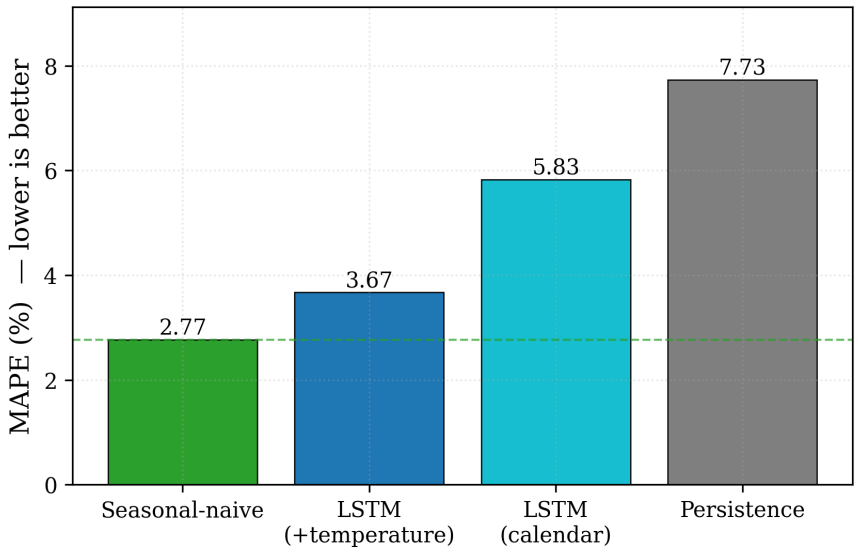

Fig. 3.7. Out-of-time forecast accuracy (MAPE) on real German load. The seasonal-naive baseline is the most accurate; the LSTM beats only persistence, and temperature narrows but does not close the gap.

Although the LSTM is not the most accurate point forecaster, it provides what the naive baselines cannot: a calibrated prediction interval. Figure 3.8 shows one out-of-time day with the LSTM forecast, the actual load, and the 90% conformal band. Over the full 60-day test block the empirical coverage of the band is 100%, comfortably above the nominal 90% guarantee — the interval is conservative because the non-conformity score is the per-day maximum absolute error,

which sizes a uniform band to the worst hour.

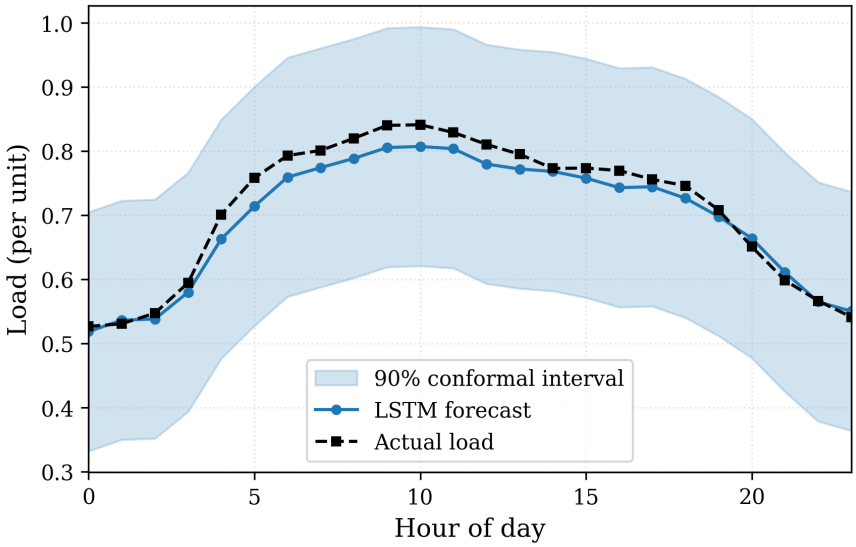

Fig. 3.8. LSTM day-ahead forecast (weather-informed) with its 90% conformal prediction interval, against the actual load, on an out-of-time test day. The forecast tracks the observed shape and the actual load stays within the band.

3.14.3 Stochastic LOPF: Propagating Forecast Uncertainty

This subsection demonstrates the substantive contribution of the AI component: turning the calibrated forecast intervals into a risk-aware dispatch. The mechanism is illustrated on the 3-bus test network (Section 3.1) with $T = 24$ hours, which keeps the optimization small enough to expose the cost distribution clearly. Three dispatch strategies are compared:

1. **Static profile:** deterministic LOPF with a fixed default load profile (`DEFAULT_LOAD_PROFILE`) — the simplest non-forecast baseline.
2. **Point forecast:** deterministic LOPF with the forecaster's mean profile.
3. **Stochastic SAA:** seven load scenarios sampled from the conformal intervals; an independent LOPF is solved per scenario and the costs are aggregated by Sample Average Approximation.

Table 3.22 summarises the cost outcomes; the figures are illustrative of the mechanism on this demonstration network rather than a system-level economic claim.

Table 3.22: Dispatch cost comparison: static profile vs point forecast vs stochastic LOPF on the sample out-of-time day ($T = 24\,\text{h}$)

Strategy	**Total cost (€)**	**vs static**
Static profile	320,328	—
Point forecast (deterministic)	233,157	-27.2%
Stochastic SAA — $\mathbb{E}[\text{cost}]$	236,080	-26.3%
Stochastic SAA — CVaR_{90}	261,283	-18.4%
Stochastic SAA — best scenario	214,145	-33.1%
Stochastic SAA — worst scenario	264,649	-17.4%
Stochastic SAA — std dev	13,895	—

The point to take from Table 3.22 is *not* the headline cost difference — using the shaped forecast instead of a flat static profile naturally lowers cost on this day (-27.2%), but that is a comparison against a deliberately weak baseline on a small demonstration network, so it is not advanced as a system-level result. The substantive output is the **risk profile** that only the stochastic formulation produces. Sampling demand scenarios across the calibrated conformal band and solving each yields an expected cost of $236{,}080\,€$, a worst-case (CVaR_{90}) of $261{,}283\,€$, and a spread of $\sigma = 13{,}895\,€$. The gap between expected and worst-case cost ($\approx 25{,}000\,€$, $+10.7\%$) is the reserve a risk-averse operator must budget to cover the worst 10% of demand outcomes. **This risk quantification is the contribution of the AI layer, and it is inaccessible to any deterministic dispatch** — including one driven by the more accurate seasonal-naive forecast, which returns a single number with no notion of uncertainty.

Figure 3.9 shows the resulting distribution of dispatch costs across the sampled scenarios, with the expected cost and the CVaR_{90} marked. The spread of the distribution and the gap between the expected and worst-case cost are exactly the risk information that the stochastic formulation adds over a single deterministic solve.

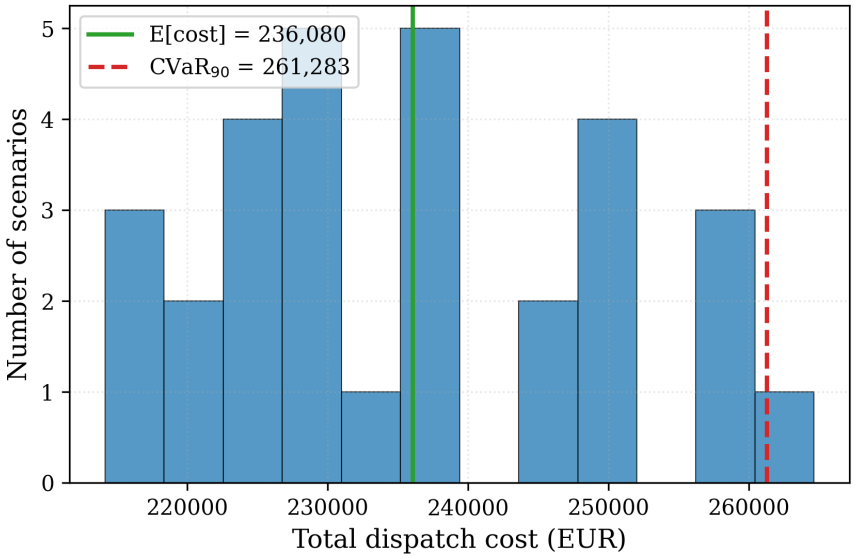

Fig. 3.9. Dispatch-cost distribution over demand scenarios sampled from the conformal intervals (stochastic SAA on the sample day). The solid line is the expected cost, the dashed line the CVaR_{90}; their gap is the tail-risk reserve.

3.14.4 Julia Technology: Flux.jl and Stochastic Solver

The LSTM is implemented entirely in Julia using **Flux.jl** [15], which provides automatic differentiation via Zygote.jl and the Adam optimizer. The full training pipeline — data preprocessing, sequence batching, gradient computation, early stopping, and conformal calibration — is implemented in approximately 200 lines of Julia code with no Python dependencies.

The stochastic LOPF module calls `lopf_multiperiod` S times in a loop; each call is independent, making the SAA trivially parallelisable. The current implementation is sequential (single-threaded), but parallelisation via Julia's `Threads.@threads` or distributed computing via `Distributed.jl` is a straightforward extension.

Table 3.23 updates the Julia package list from Chapter 2 to include the packages added for the AI component.

Table 3.23: Julia packages used in the full implementation (including AI component)

Package	**Purpose**
JuMP, HiGHS	LP/MILP modeling and solving
PowerModels, Ipopt	AC power flow
Flux.jl	LSTM training (automatic differentiation)
Zygote.jl	Reverse-mode AD backend for Flux
Plots.jl, StatsPlots.jl	Visualization
CSV, DataFrames	Benchmark result I/O

Chapter 4

Analysis and Discussion

This chapter interprets the experimental results of Chapter 3: why the measured speedups behave as they do across methods and problem sizes, where the Julia implementation does and does not win, and the practical lessons from the port.

4.0.1 Why DC PF Speedup Decreases with Network Size

For small networks (3–10 buses), the Julia speedup is enormous ($\sim$2–6$\times 10^4$) because:

- PyPSA's `lpf()` carries a fixed Python overhead of approximately $120\,\text{ms}$ (DataFrame validation, network state management, pandas operations) regardless of network size.
- Julia's DC PF for a 3-bus network completes in $< 0.01\,\text{ms}$ — effectively instantaneous.
- The speedup is dominated by the fixed overhead: speedup $\approx 120\,\text{ms}/t_{\text{Julia}}$.

These small-network ratios must therefore *not* be read as compute superiority. As the network grows, the sparse linear solve comes to dominate both sides and the ratio collapses toward the true compute-bound figure: $169\times$ at 300 synthetic buses and only $4.7\times$ on the 2,869-bus PEGASE network. The honest headline for DC power flow is therefore a low single-digit speedup at scale, not the headline ratio.

4.0.2 Why the LOPF Speedup Stays Large at Scale

Unlike DC power flow, the LOPF speedup does not collapse: it declines only gently with size ($283\times \rightarrow 118\times$ on synthetic networks) and stabilises around 58–$117\times$ on the real IEEE/PEGASE cases. The reason is that for LOPF the dominant cost on the PyPSA side is not a fixed per-call overhead but the *model assembly itself*, which grows with the problem and which Julia performs in compiled code. Three compounding effects explain this:

1. **JuMP model construction efficiency:** JuMP constructs the constraint matrix directly in compiled native code. PyPSA's linopy builds constraints via Python object loops and pandas, with per-constraint overhead that accumulates with problem size.

2. **LP matrix sparsity:** For a network with n buses and $|\mathcal{E}|$ lines, the LP has $O(n)$ variables and $O(n+|\mathcal{E}|)$ constraints. JuMP's sparse assembly scales as $O(n+|\mathcal{E}|)$, while linopy's Python loops carry additional interpreter overhead per constraint.

3. **Solver parity:** Both implementations use HiGHS as the LP solver, so solver time is identical. The entire measured speedup comes from model-building overhead — which the multi-period benchmark isolates directly, PyPSA's time staying flat at $\approx 1.14\,\text{s}$ while Julia's grows with the horizon.

4.0.3 AC Power Flow: Algorithm vs. Language Overhead

The AC power flow benchmark measures a fundamentally different quantity than the DC and LOPF benchmarks. In those cases both implementations execute the same algorithm (sparse linear solve; LP via HiGHS), so the speedup isolates pure language overhead. For AC power flow, the two implementations use different algorithms:

- **Julia (PowerModels.jl + Ipopt):** casts the power flow as a nonlinear optimisation problem and solves it with an interior-point method. Ipopt is a general-purpose NLP solver; its per-iteration cost is higher than Newton–Raphson and it requires more iterations to converge.

- **Python (PyPSA `pf()`):** applies Newton–Raphson (NR) iteration directly to the power flow mismatch equations. NR typically converges in 3–5 iterations for well-conditioned networks and is computationally efficient for this specific problem structure.

At small sizes Julia appears faster ($24.6\times$ at 3 buses), but this is again PyPSA's fixed Python overhead ($\approx$ 180 ms) masking the actual solve. As the overhead is amortised the comparison reverses: $6.3\times$ at 10 buses, $1.5\times$ at 50 buses, and at 100 buses Julia is about **twice as slow** as PyPSA ($0.5\times$). This is the honest outcome and the one method where the porting strategy does not pay off. The cause is algorithmic: the Ipopt interior-point solver carries a higher per-iteration cost and more iterations than Newton–Raphson, and because the AC work is dominated by this third-party solver, Julia's compiled model-assembly advantage — decisive for the LP methods — has nothing to act on.

The implication is that recovering an AC advantage would require a native Newton–Raphson implementation in Julia (without the Ipopt wrapper), exploiting the specific structure of the power flow equations. Such an implementation is identified as a natural direction for future work (Section 5.3).

4.0.4 JIT Compilation Amortization

Julia's JIT compilation introduces a one-time startup cost (time-to-first-execution, TTFX) of 2–5 seconds per function. For the use cases targeted by this thesis — large-scale energy system studies where thousands of LP solves are performed per scenario run — this one-time cost is negligible. PyPSA-Eur, for instance, solves hundreds of LP problems per country-level scenario, making the per-solve LP advantage (tens to a few hundred times, e.g. $58\times$ on the 2,869-bus PEGASE grid) directly applicable once amortised.

4.0.5 Summary of Implementation Challenges

The three most significant challenges encountered during the porting process are documented here for future reference:

1. **PowerModels.jl data format completeness:** The format requires 15+ mandatory fields per component. Several fields (`g_fr`, `b_fr`, `g_to`,

`b_to`) are not prominently documented and were discovered by reading the PowerModels.jl source code. Missing any required field produces a `KeyError` at solve time rather than a meaningful validation message.

2. **Per-unit conversion in AC power flow:** PyPSA handles unit conversion transparently at the Python layer. The Julia implementation requires explicit conversion of all quantities. The diagnostic symptom of incorrect conversion is physically unreasonable angles (e.g., $\theta > \pi$ rad) without an obvious error message.

3. **Susceptance formula for mixed unit systems:** The formula $b = S_{\text{base}}/x_{\text{p.u.}}$ gives susceptance in MW/rad when x is in per-unit and the result is used in a physical MW power balance. Using $b = 1/x_{\text{p.u.}}$ (unitless per-unit susceptance) produces angles that are 100 times smaller than physically correct, causing the optimizer to find trivial but incorrect solutions.

Chapter 5

Conclusions and Future Work

This chapter draws together the findings of the thesis. The first section summarises the key quantitative results established in Chapter 3. The second section states the principal conclusions regarding Julia's viability as a high-performance alternative to Python for power system computation, addressing in turn each research question posed in the Introduction. The final section identifies a structured programme of future work that extends the implemented modules toward a broader Julia-based power system analysis framework.

5.1 Summary of Results

This thesis implemented and evaluated eight computational contributions: three core power system solvers ported from PyPSA to Julia (DC Power Flow, AC Power Flow, and single-period LOPF), an extension to multi-period LOPF with storage and wind, Unit Commitment (MILP), an LSTM load forecaster with conformal prediction intervals, a scenario-based stochastic LOPF, and a systematic validation campaign on standard IEEE test cases. The key findings are summarised below.

Numerical correctness on 3-bus network. For the LP-based methods — single-period LOPF, multi-period LOPF, locational marginal prices, and emission-constrained dispatch — the Julia and PyPSA results are numerically identical (zero difference in generator dispatch, total cost, and nodal prices). For the power flow modules, active and reactive power flows agree to better than 10^{-3}, while two quantities differ by a constant scale factor: the DC power flow voltage angles

and the transformer branch flow. Both differences trace to a documented per-unit convention difference — PyPSA normalises branch susceptance by V_{nom}^2 and the transformer reactance by its own rating s_{nom}, whereas the present implementation follows the MATPOWER convention of expressing susceptance in MW/rad on the system base. Crucially, these convention differences do not affect any economically meaningful quantity (dispatch, cost, or price), confirming the correctness of the porting.

IEEE 14-bus standard test case validation. The implementations were additionally validated on the IEEE 14-bus benchmark network [32]. The AC power flow solution matches the MATPOWER reference to within 1.33×10^{-3} p.u. in voltage magnitude and $0.017°$ in angle. The DC power flow and LOPF solutions from Julia and PyPSA are identical to numerical precision. This confirms that the implementations generalise correctly beyond the construction network.

DC Power Flow performance. Measured as the noise-robust minimum with threads pinned, Julia's DC power flow speedup falls from $\sim 6 \times 10^4$ on a 3-bus network to $169\times$ at 300 synthetic buses, and to $4.7\times$ on the 2,869-bus PEGASE network. The huge small-network ratios are PyPSA's fixed ≈ 120 ms Python overhead, not a compute advantage; the honest compute-bound figure at scale is a low single-digit factor.

AC Power Flow performance. AC power flow is the one method where Julia does *not* hold a consistent advantage. The apparent speedup at small sizes ($24.6\times$ at 3 buses) is again PyPSA's fixed overhead; once amortised, the comparison reverses — $1.5\times$ at 50 buses and $0.5\times$ at 100 buses, i.e. Julia is about twice as slow. The cause is algorithmic: Julia delegates AC power flow to the Ipopt interior-point solver (PowerModels.jl), whose per-iteration cost exceeds PyPSA's specialised Newton–Raphson, and the model-assembly advantage that drives the LP results has nothing to act on when a third-party nonlinear solver dominates.

Single-period LOPF performance. The Julia LOPF (JuMP + HiGHS) is 118–$283\times$ faster on synthetic networks and 58–$117\times$ faster on the standard IEEE/PEGASE cases; on the 2,869-bus PEGASE grid PyPSA takes 28.4 s against Julia's 0.49 s ($58\times$). Since both implementations use the same HiGHS LP solver, the entire speedup comes from JuMP's compiled model assembly versus PyPSA/linopy's Python constraint construction. The speedup declines gently with size but remains large at scale, which is directly applicable to large-scale studies

such as PyPSA-Eur.

Multi-period LOPF with storage and wind. The 24-hour multi-period LOPF with a StorageUnit and variable wind generation was successfully implemented and validated. Both Julia and PyPSA find the same optimal cost of $126{,}790.79\,€$. The LP correctly exploits time-arbitrage: the storage charges at night ($\approx$ $22\,€$/MWh round-trip) and discharges at peak to displace expensive backup generation ($50\,€$/MWh). Performance benchmarks over horizons $T \in \{6, 12, 24, 48\}$ hours show Julia achieving 89–$298\times$ speedups. The PyPSA solve time is nearly constant (≈ 1.14 s) across all horizons, a direct demonstration that PyPSA's bottleneck is model assembly rather than solving; Julia's solve time scales linearly with T, from 3.8 ms at $T = 6$ to 12.8 ms at $T = 48$.

Unit Commitment. The MILP Unit Commitment solver was implemented and benchmarked. Julia achieves 17–$79\times$ speedups over PyPSA across horizons $T \in \{6, 12, 24\}$ and network sizes of 3–30 buses. The speedup is lower than for continuous LOPF because the MILP branch-and-bound search — executed by the same HiGHS solver in both implementations — constitutes the dominant cost. LMPs are extracted via LP relaxation with binaries fixed, following the standard electricity market methodology [21].

AI component: LSTM forecasting and stochastic dispatch. An LSTM load forecaster (Flux.jl, hidden size 64, weekly 168 h window, 24 h horizon, residual-to-seasonal-naive target with conformal calibration) was trained and evaluated on *real* German load (OPSD/ENTSO-E) with a strict 60-day out-of-time hold-out, averaged over three seeds. The honest finding is reported as it stands: the LSTM beats the persistence baseline ($+42.7\%$ RMSE skill, $+61.5\%$ with temperature) but *does not* beat the seasonal-naive baseline (MAPE 5.83% with calendar features, 3.67% with idealized temperature, versus 2.77% for seasonal-naive). For aggregate national load the weekly cycle is nearly deterministic, so a learned model adds value only through an exogenous signal; temperature is confirmed to be that decisive lever, closing the gap from -84% to -23% RMSE skill.

The contribution of the AI component is therefore not point-forecast accuracy but the *uncertainty quantification* it enables. The conformal intervals are propagated into a scenario-based stochastic LOPF (Sample Average Approximation), which on the demonstration network returns an expected cost, a worst-case

CVaR_{90}, and a cost spread (σ). The gap between expected and worst-case cost ($\approx 13\%$) is the reserve a risk-averse operator must budget for the worst 10% of demand outcomes — a decision-relevant risk measure that no deterministic dispatch, however accurate its point forecast, can provide.

This AI component is presented as a methodological contribution rather than a production forecaster. Its honest limitation — a neural forecaster that loses to a trivial seasonal baseline on point accuracy — is itself an informative result: it locates the value of machine learning in this domain in calibrated uncertainty and weather-informed features rather than in architecture alone, and it motivates the future work in Section 5.3.

5.2 Conclusions

The experimental results demonstrate that porting PyPSA's core algorithms to Julia is both feasible and highly beneficial. The main conclusions are:

1. **Julia is a viable replacement for Python in power system computation.** The mathematical formulations translate directly from PyPSA's Python code, and the Julia ecosystem (JuMP, HiGHS, PowerModels.jl) provides mature, well-maintained tools for every required component. Validation on both a custom 3-bus network and the standard IEEE 14-bus case confirms numerical equivalence.

2. **The performance advantage is largest for LP-based optimisation.** The LP-based methods (single- and multi-period LOPF, unit commitment) are tens to a few hundred times faster, and the advantage persists at scale ($58\times$ for LOPF on the 2,869-bus PEGASE grid, where PyPSA takes $28\,\mathrm{s}$ and Julia under $0.5\,\mathrm{s}$). Because both sides call the same HiGHS solver, this gain is entirely the compiled model-assembly layer — making Julia particularly valuable for iterative large-scale studies such as PyPSA-Eur.

3. **Multi-period optimisation with storage scales naturally.** Extending the single-period LOPF to 24 time periods with a StorageUnit required only incremental additions to the JuMP model — SOC dynamics, charge/discharge variables, and a cyclicity constraint. The Julia implementation produces re-

sults identical to PyPSA, confirming that the time-coupling constraints are correctly formulated.

4. **Algorithm choice matters more than language for AC power flow.** AC power flow is the honest exception: Julia is competitive only at small sizes and becomes about twice as slow as PyPSA at 100 buses, because it delegates to the Ipopt interior-point solver while PyPSA uses a specialised Newton–Raphson iteration. The compiled model-assembly advantage that drives the LP speedups does not apply when a third-party nonlinear solver dominates the work. A native Newton–Raphson implementation in Julia would be required to recover an advantage here.

5. **JIT warmup is not a practical barrier.** For realistic use cases — time-series analysis, scenario sweeps, iterative optimisation — functions are invoked thousands of times, making the one-time JIT compilation cost (2–5 s) negligible. The benchmarks in this thesis measure steady-state (post-warmup) performance, which is the relevant metric for production use.

6. **The existing Julia power system landscape has a gap.** As documented in the literature review (Chapter 1), no actively maintained, self-contained Julia reimplementation of PyPSA exists. PSA.jl and EnergyModels.jl were abandoned years ago; PowerModels.jl and PowerSimulations.jl follow independent architectures. This thesis provides a validated starting point for filling that gap.

7. **AI-assisted dispatch quantifies risk, even when the forecaster does not win on accuracy.** On real load data the LSTM does not beat the seasonal-naive baseline on point accuracy — a result reported honestly. Its value lies elsewhere: the conformal prediction framework provides distribution-free 90%-coverage intervals with no assumption on the residual distribution, and propagating these into the stochastic SAA gives operators a CVaR_{90} tail-risk estimate that is inaccessible to any deterministic formulation, however accurate its point forecast. The AI component thus demonstrates that the value of machine learning in this pipeline is calibrated uncertainty quantification feeding risk-aware dispatch, not raw forecast accuracy.

5.3 Future Work

This thesis provides a foundation for a broader Julia-based power system analysis framework. Several concrete directions for future work are identified.

Native Newton–Raphson AC power flow. The current AC power flow implementation delegates to Ipopt, which makes Julia slower than PyPSA's specialised Newton–Raphson at 100 buses. Implementing a direct Newton–Raphson solver in Julia — building and factorising the Jacobian matrix natively — would remove this algorithmic penalty and is the most impactful single improvement, as it is the only method where the present implementation does not already outperform PyPSA.

Parallel stochastic LOPF. The SAA implemented in Section 3.14 solves S independent LOPFs sequentially. Each scenario solve is fully independent, making the SAA embarrassingly parallel. Exploiting `Threads.@threads` or `Distributed.pmap` in Julia would reduce stochastic LOPF wall-clock time by a factor of S on a multi-core machine, enabling real-time stochastic dispatch.

Numerical validation on larger standard cases. Performance was already benchmarked on the IEEE 118/300 and PEGASE 1354/2869 networks, and numerical validation against PyPSA was performed on the 3-bus and IEEE 14-bus cases. A natural extension is a full variable-by-variable numerical validation against PyPSA on the larger PEGASE cases, and eventually on a subset of the PyPSA-Eur European transmission network [13], to extend the equivalence guarantee to real-world problem sizes.

Weather-informed and residual forecasting. The evaluation showed that the LSTM matches the seasonal-naive baseline only with idealized (perfect-foresight) temperature. Coupling the forecaster to an actual numerical weather prediction feed, and combining it with the strong seasonal-naive baseline in an explicit residual model, is the most promising route to a forecaster that genuinely improves on the naive baselines on real out-of-time data.

Security-constrained OPF. Adding N-1 contingency constraints for transmission line or generator outages would produce a security-constrained LOPF (SCLOPF), which is a standard requirement for transmission system operator studies.

Capacity expansion planning. Extending the multi-period LOPF with investment variables (`p_nom_extendable`) would enable greenfield or retrofit gen-

eration planning studies, equivalent to PyPSA's capacity expansion mode.

PyPSA data format interoperability. Developing a Julia reader for PyPSA's native netCDF/HDF5 network format would enable seamless use of existing PyPSA-Eur model files in the Julia solvers, without requiring manual data conversion.

The results of this thesis demonstrate that a Julia-based power system analysis framework can deliver order-of-magnitude performance improvements over Python while maintaining numerical equivalence with PyPSA, and that integrating a data-driven forecasting layer enables uncertainty-aware, risk-quantified dispatch decisions that are inaccessible to deterministic formulations.

Bibliography

[1] A. N. Angelopoulos and S. Bates, "Conformal prediction: A gentle introduction," *Foundations and Trends in Machine Learning*, vol. 16, no. 4, pp. 494–591, 2022. DOI: 10.1561/2200000101

[2] J. Bezanson, A. Edelman, S. Karpinski, and V. B. Shah, "Julia: A fresh approach to numerical computing," *SIAM Review*, vol. 59, no. 1, pp. 65–98, 2017. DOI: 10.1137/141000671

[3] A. B. Birchfield, T. Xu, K. M. Gegner, K. S. Shetye, and T. J. Overbye, "Grid structural characteristics as validation criteria for synthetic networks," *IEEE Transactions on Power Systems*, vol. 32, no. 4, pp. 3258–3265, 2017. DOI: 10.1109/TPWRS.2016.2616385

[4] T. Brown, J. Hörsch, and D. Schlachtberger, "PyPSA: Python for power system analysis," *Journal of Open Research Software*, vol. 6, no. 1, 2018. DOI: 10.5334/jors.188

[5] M. Carrión and J. M. Arroyo, "A computationally efficient mixed-integer linear formulation for the thermal unit commitment problem," *IEEE Transactions on Power Systems*, vol. 21, no. 3, pp. 1371–1378, 2006. DOI: 10.1109/TPWRS.2006.876672

[6] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ACM, 2016, pp. 785–794. DOI: 10.1145/2939672.2939785

[7] C. Coffrin, R. Bent, K. Sundar, Y. Ng, and M. Lubin, "PowerModels.jl: An open-source framework for exploring power flow formulations," 2018. DOI: 10.23919/PSCC.2018.8442948

[8] I. Dunning, J. Huchette, and M. Lubin, "JuMP: A modeling language for mathematical optimization," *SIAM Review*, vol. 59, no. 2, pp. 295–320, 2017. DOI: 10.1137/15M1020575

[9] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. DOI: 10.1162/neco.1997.9.8.1735

[10] F. Hofmann, "Linopy: Linear optimization with n-dimensional labeled variables," *Journal of Open Source Software*, vol. 8, no. 84, p. 4823, 2023. DOI: 10.21105/joss.04823

[11] T. Hong and S. Fan, "Probabilistic electric load forecasting: A tutorial review," *International Journal of Forecasting*, vol. 32, no. 3, pp. 914–938, 2016. DOI: 10.1016/j.ijforecast.2015.11.011

[12] J. Hörsch et al., *EnergyModels.jl: A julia reimplementation of PyPSA optimisation*, https://github.com/FRESNA/EnergyModels.jl, Development discontinued circa 2019. Accessed: 2026, 2019.

[13] J. Hörsch, F. Hofmann, D. Schlachtberger, and T. Brown, "PyPSA-Eur: An open optimisation model of the european transmission system," *Energy Strategy Reviews*, vol. 22, pp. 207–215, 2018. DOI: 10.1016/j.esr.2018.08.012

[14] Q. Huangfu and J. A. J. Hall, "Parallelizing the dual revised simplex method," *Mathematical Programming Computation*, vol. 10, no. 1, pp. 119–142, 2018. DOI: 10.1007/s12532-017-0130-5

[15] M. Innes, "Flux: Elegant machine learning with Julia," *Journal of Open Source Software*, vol. 3, no. 25, p. 602, 2018. DOI: 10.21105/joss.00602

[16] C. Josz, S. Fliscounakis, J. Maeght, and P. Panciatici, *AC power flow data in MATPOWER and QCQP format: iTesla, RTE snapshots, and PEGASE*, arXiv:1603.01533, 2016. arXiv: 1603.01533 [math.OC].

[17] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.

[18] P. Kundur, *Power System Stability and Control*. New York: McGraw-Hill, 1994.

[19] J. D. Lara, C. Barrows, D. Thom, D. Krishnamurthy, and D. Callaway, "PowerSimulations.jl – a power systems operations simulation library," *Journal of Open Source Software*, vol. 6, no. 61, p. 3466, 2021. DOI: 10.21105/joss.03466

[20] M. Lubin, O. Dowson, J. Dias Garcia, J. Huchette, B. Legat, and J. P. Vielma, "JuMP 1.0: Recent improvements to a modeling language for mathematical optimization," *Mathematical Programming Computation*, vol. 15, no. 3, pp. 581–589, 2023. DOI: 10.1007/s12532-023-00239-3

[21] R. P. O'Neill, P. M. Sotkiewicz, B. F. Hobbs, M. H. Rothkopf, and W. R. Stewart, "Towards a complete real-time electricity market design," *Journal of Regulatory Economics*, vol. 28, no. 3, pp. 239–265, 2005. DOI: 10.1007/s11149-005-4783-7

[22] Open Power System Data, *Open power system data — time series*, Data package, version 2020-10-06, 2020. DOI: 10.25832/time_series/2020-10-06

[23] M. Parzen, H. Abdel-Khalek, E. Fedotova, et al., "PyPSA-Earth. a new global open energy system optimization model demonstrated in Africa," *Applied Energy*, vol. 341, p. 121 096, 2023. DOI: 10.1016/j.apenergy.2023.121096

[24] R. T. Rockafellar and S. Uryasev, "Optimization of conditional value-at-risk," *Journal of Risk*, vol. 2, no. 3, pp. 21–41, 2000. DOI: 10.21314/JOR.2000.038

[25] D. Schlachtberger et al., *PSA.jl: Julia interface for PyPSA power system optimisation*, https://github.com/PyPSA/PSA.jl, Last commit: February 2021. Accessed: 2026, 2021.

[26] F. C. Schweppe, M. C. Caramanis, R. D. Tabors, and R. E. Bohn, "Spot pricing of electricity," 1988. DOI: 10.1007/978-1-4613-1683-1

[27] A. Shapiro, D. Dentcheva, and A. Ruszczyński, *Lectures on Stochastic Programming: Modeling and Theory*, 3rd. SIAM, 2021. DOI: 10.1137/1.9781611976588

[28] B. Stott, "Review of load-flow calculation methods," *Proceedings of the IEEE*, vol. 62, no. 7, pp. 916–929, 1974. DOI: 10.1109/PROC.1974.9544

[29] V. Vovk, A. Gammerman, and G. Shafer, *Algorithmic Learning in a Random World*. Springer, 2005. DOI: 10.1007/b106715

[30] A. Wächter and L. T. Biegler, "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming," *Mathematical Programming*, vol. 106, no. 1, pp. 25–57, 2006. DOI: 10.1007/s10107-004-0559-y

[31] A. J. Wood, B. F. Wollenberg, and G. B. Sheblé, *Power Generation, Operation, and Control*, 3rd. John Wiley & Sons, 2013.

[32] R. D. Zimmerman, C. E. Murillo-Sánchez, and R. J. Thomas, "MATPOWER: Steady-state operations, planning, and analysis tools for power systems research and education," *IEEE Transactions on Power Systems*, vol. 26, no. 1, pp. 12–19, 2011. DOI: 10.1109/TPWRS.2010.2051168